



A linear-time algorithm to compute the conjugate of nonconvex bivariate piecewise linear-quadratic functions

Tanmaya Karmarkar¹ · Yves Lucet¹ 

Received: 9 October 2024 / Accepted: 9 May 2025 / Published online: 27 May 2025

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract

We propose the first linear-time algorithm to compute the conjugate of (nonconvex) bivariate piecewise linear-quadratic (PLQ) functions (bivariate quadratic functions defined on a polyhedral subdivision). Our algorithm starts with computing the convex envelope of each quadratic piece obtaining rational functions (quadratic over linear) defined over a polyhedral subdivision. Then we compute the conjugate of each resulting piece to obtain piecewise quadratic functions defined over a parabolic subdivision. Finally we compute the maximum of all those functions to obtain the conjugate as a piecewise quadratic function defined on a parabolic subdivision. The resulting algorithm runs in linear time if the initial subdivision is a triangulation (or has a uniform upper bound on the number of vertices for each piece). Our open-source implementation in MATLAB uses symbolic computation and rational numbers to avoid floating-point errors, and merges pieces as soon as possible to minimize computation time.

Keywords Global optimization · Conjugate · Convex envelope · Piecewise quadratic function

Mathematics Subject Classification 90C25 · 65K10 · 49M29 · 26B25

1 Introduction

There are two motivations for the present work: obtaining tighter lower bounds for relaxation in global optimization, and computing the conjugate in computational convex analysis. Beyond applications, we want to understand the structure of the conjugate of piecewise linear-quadratic (PLQ) functions (bivariate functions defined on a union of polyhedral set on each of which the restriction of the function is quadratic). Understanding the structure is of interest in itself; it is also of interest to make progress toward computing the convex envelope

Tanmaya Karmarkar and Yves Lucet contributed equally to this work

✉ Yves Lucet
yves.lucet@ubc.ca

Tanmaya Karmarkar
tanmayak@student.ubc.ca

¹ Computer Science, CMPS, I. K. Barber Faculty of Science, UBC Okanagan, 3187 University Way, V1T 1T7 Kelowna, BC, Canada

(computing the conjugate is the first step toward computing the biconjugate, which is the convex envelope). The convex envelope provides the tightest relaxation when one wishes to solve a global optimization problem (the set of global minima of a function is included in the set of minima of its convex envelope). A large literature is available on global optimization and the search for tight relaxation; we refer to [1] for a detailed introduction and more details on relaxations relevant for our context.

Computing the convex envelope of a function is a hard problem; even computing the convex envelope of a multilinear function over a unit hypercube is NP-Hard [2]. However, results for specific functions exist in the literature, in particular for quadratic bivariate polynomials [3–7], and for convex envelopes of bilinear functions over triangles, rectangles and special polytopes [8–12]. It is an active subject of research [13].

PLQ functions play a significant role in variational analysis [14, Section 10E, Section 11D, p. 440] due to the availability of calculus rules [14, Example 11.28, Proposition 11.32, Corollary 11.33, Proposition 12.30], and to their duality property [14, Theorem 11.42, Example 11.43, Theorem 11.42]. Compared to piecewise linear functions, PLQ functions capture the curve of the original function more accurately. The set of convex PLQ functions is closed under common convex operators, in particular under the Legendre-Fenchel transform; see [14, Page 484], [15, Proposition 3, p. 17] and [16].

The early idea of the computation of convex transforms can be traced back to [17]. However, development of most of the algorithms in computational convex analysis began with the computation of the conjugate with the Fast Legendre Transform (FLT) [18], which is studied in [19, 20]. A linear-time algorithm is introduced in [21]. Those algorithms handle nonconvex functions but are restricted to grid domains. Extension to nongrid domains for convex functions only are proposed in [16, 22] with a generalization to the partial conjugate in [16]. Computation of the conjugate of convex univariate PLQ functions has been well studied and linear time algorithms have been developed, both for the PLQ [23] and the graph-matrix (GPH) models [24]. All in all, there is no algorithm to compute the conjugate of nonconvex PLQ functions over nongrid domains.

Implementation of conjugate computation algorithms can be found in the CCA library [25, 26] that contains efficient algorithms to manipulate convex functions and to compute fundamental convex analysis transforms arising in the field of convex analysis. Algorithms to compute the conjugate numerically on grids have been based on either parameterization [27], manipulation of graphs (GPH model) [16], or the computation of the Moreau envelope [28]. More complex operators such as the proximal average operator [29, 30] can be built by using a combination of addition, scalar multiplication, and conjugacy operations.

A numerical library to determine in linear time whether a PLQ function is convex [31] is available in MATLAB [32]. This library is the first to handle general piecewise quadratic functions that are not necessarily continuous and are defined on any polyhedral subdivision without approximating them with a grid.

Computational Convex Analysis has many applications in the fields of image processing, network communication, PDE, geographic information systems, computer aided design, molecular biology, medical imaging, computer graphics and robotics; see the survey [33].

We propose the first algorithm to compute the conjugate of a general bivariate PLQ function not necessarily convex without approximating it with a grid. Our algorithm is split in three steps. In step 1, we apply [5] to compute the convex envelope of each piece; in step 2, we apply [34] to compute the conjugate of each resulting piece; and in step 3, we compute the maximum of all the resulting piecewise functions.

Our contribution is fourfold: (1) proving that the conjugate is a piecewise quadratic function defined on a parabolic subdivision (Kumar [34] had in addition a fractional form; we

prove that such fractional form cannot occur); (2) performing the very last step to obtain the formula for the conjugate (that step was not performed in [34]), and proving that computing the maximum results in a piecewise quadratic function defined on a parabolic subdivision; (3) proving the entire algorithm runs in linear time; and (4) releasing an open-source code that includes numerous techniques to minimize computation time (merging adjacent pieces with equal functions using symbolic computation).

We begin by giving some preliminaries in Section 2 and the overall algorithm in Section 3.1. We present examples in Section 4, and conclude with future work in Section 5.

2 Preliminaries

The domain of a function $f : \mathbb{R}^n \rightarrow \mathbb{R} \cup \{+\infty\}$ is the set $\{x : f(x) < +\infty\}$, and the function is called proper if it has nonempty domain. The indicator function of $C \subset \mathbb{R}^n$ is denoted $I_C : \mathbb{R}^n \rightarrow (-\infty, \infty]$ with $I_C(x) = 0$ if $x \in C$ and $+\infty$ otherwise.

A function is called a piecewise function if it can be written $f(x) = f_i(x)$ for $x \in R_i$ with $\cup_i R_i = \mathbb{R}^n$. We call the pair (f_i, R_i) a piece. Recalling [14, Definition 10.20], a function $f : \mathbb{R}^n \rightarrow \mathbb{R} \cup \{+\infty\}$ is called Piecewise Linear-Quadratic (PLQ) if $\text{dom } f = \cup_i R_i$ and f restricted to R_i , noted f_i can be written $f_i(x) = 1/2x^T Qx + q^T x + \kappa$ with $\kappa \in \mathbb{R}$, $q \in \mathbb{R}^n$, and $Q \in \mathbb{R}^{n \times n}$ a symmetric matrix. We use column vectors with $q^T x$ denoting the dot product of row vector q^T with column vector x . PLQ functions enjoy several properties, e.g., their domain is closed, they are lower semicontinuous, and continuous relative to their domain [14, Proposition 10.21].

Example 1 (Univariate PLQ function.) The function $f(x) = x^2$ if $x \leq 0$, $f(x) = 1 - (1-x)^2$ if $0 < x < 1$, and $f(x) = x^2$ when $x \geq 1$ is an example of a univariate PQ function.

Example 2 (Bivariate PLQ function.) The function

$$\begin{aligned} f(x, y) &= 2x^2 - xy - y^2, & (x, y) &\in \text{conv}((2, 1), (3, 5), (6, 3)), \\ f(x, y) &= x^2 + xy - y^2, & (x, y) &\in \text{conv}((2, 1), (6, 3), (4, 0)), \end{aligned}$$

is an example of a PLQ function in two variables.

An underestimator is any function that lies below a given function. The closed convex envelope is the largest lower semicontinuous convex underestimator of f .

Example 3 (Convex Envelope of Example 1) The convex envelope $\text{conv } f$ of the function f defined in Example 1 is $\text{conv } f(x) = x^2$ if $x \leq 0$, $\text{conv } f(x) = x$ if $0 < x < 1$, and $\text{conv } f(x) = x^2$ if $x \geq 1$.

We note

$$f^*(y) = \sup_{x \in \mathbb{R}^n} (x^T y - f(x))$$

the (Legendre-Fenchel) conjugate of f . The biconjugate is noted f^{**} , and we recall that when f is proper, closed and convex, $f = f^{**}$, otherwise f^{**} is the closed convex envelope of f [14, Theorem 11.1].

To describe the domain of piecewise functions, we note a hyperplane $H \subset \mathbb{R}^d$ a set of the form $H = \{x \in \mathbb{R}^d : \alpha^T x - \beta = 0\}$, and a halfspace $H \subset \mathbb{R}^d$ a set of the form

$H = \{x \in \mathbb{R}^d : \alpha^T x - \beta \leq 0\}$ or $H = \{x \in \mathbb{R}^d : \alpha^T x - \beta \geq 0\}$; where the vector $\alpha \in \mathbb{R}^d \setminus \{0\}$ is called a normal vector to H .

A polytope is a convex combination of finite numbers of vertices $\{v_1, \dots, v_k\}$. We write $\text{conv } V = \{\lambda_1 v_1 + \dots + \lambda_k v_k : \lambda_i \geq 0, \sum_{i=1}^k \lambda_i = 1\}$ or in matrix notation $\text{conv } V = \{V\lambda : \lambda_i \geq 0, \sum_{i=1}^k \lambda_i = 1\}$, where V is the matrix formed by column vectors. Note that by our definition, polytopes are always bounded.

A polyhedral cone is defined as a conic combination of a finite numbers of directions $\{d_1, \dots, d_m\}$; we write $\text{cone } C = \{\mu_1 d_1 + \dots + \mu_m d_m : \mu_i \geq 0\}$, and $\text{cone } C = \{D\mu : \mu_i \geq 0\}$; where D is the matrix formed by column vectors d_i .

A polyhedral set (polyhedron) $P \subset \mathbb{R}^d$ is defined as the intersection of a finite number of closed halfspaces and hyperplanes; it has the H -representation

$$P = \{x \in \mathbb{R}^d : Ax \leq b\}$$

with $A \in \mathbb{R}^{m \times d}$ and $b \in \mathbb{R}^m$. We can also describe a polyhedral set in Vertex-representation. According to the Minkowski-Weyl Theorem [14, Corollary 3.53], a polyhedral set can be written as a sum of a polytope and a polyhedral cone. Therefore, in V -representation a polyhedral set P is described as

$$P = \text{conv } V + \text{cone } C = \left\{ V\lambda + D\mu : \lambda \in \mathbb{R}^k, \lambda \geq 0, \mu \geq 0, \sum_{i=1}^k \lambda_i = 1 \right\}.$$

Since the intersection of convex sets is convex, polyhedral sets are convex.

A d -dimensional face of a convex set $C \subset \mathbb{R}^n$ is a d -dimensional convex subset C' of C such that every (closed) line segment in C with a relative interior point in C' has both endpoints in C' [35, Page 162]. The empty set and C itself are faces of C . The two-dimensional faces of C are called faces. The one-dimensional faces of a convex set C are called edges. For $u, v \in \mathbb{R}^d, u \neq v$ and $\delta = (\delta_1, \delta_2) \in \Delta \subset \mathbb{R}^2$, an edge can be written as

$$\left\{ x \in \mathbb{R}^d : x = \delta_1 u + \delta_2 v, \delta_1 + \delta_2 = 1 \right\}.$$

An edge in $\mathbb{R}^2$ is a segment if $\Delta = \mathbb{R}_+ \times \mathbb{R}_+$ where $\mathbb{R}_+ = \{\alpha \in \mathbb{R} : \alpha \geq 0\}$, a ray if $\Delta = \mathbb{R}_+ \times \mathbb{R}$, and a line if $\Delta = \mathbb{R}^2$. The zero-dimensional faces of a convex set C are called vertices and are the extreme points of C .

A convex set defined as the union of a finite number of polyhedral regions, namely $R = \bigcup_{i=1}^n R_i$, where $R \subseteq \mathbb{R}^2$, is said to be a polyhedral subdivision [36, Definition 1], if R_i is a polyhedral set and for any $j, k \in \{1, \dots, n\}, j \neq k, R_j \cap R_k$ is either empty, a vertex or an edge. (Polyhedral subdivisions eliminate degenerate subdivisions for which the intersection of two polyhedral regions is a strict subset of an edge, or of a face.)

Assume that $f : \mathbb{R}^2 \rightarrow \mathbb{R} \cup \{+\infty\}$ is a PLQ function and that $\text{dom } f = \bigcup_i P_i$ is a polyhedral subdivision. An entity is a d -dimensional face of P_i for some index i . An entity is either a vertex, an edge or a face of the polyhedral subdivision of $\text{dom } f$ [37].

We now define the geometric shapes required to describe the domain of the conjugate functions we obtain. A parabola $P \subset \mathbb{R}^2$ is a subset of the plane that can be written as

$$P = \{(x, y) \in \mathbb{R}^2 : ax^2 + bxy + cy^2 + dx + ey + f = 0\},$$

where $a, b, c, d, e, f \in \mathbb{R}$ are not all zero and satisfy $b^2 - 4ac = 0$. Note that our definition includes lines when $a = b = c = 0$, and the empty set when $a = b = c = d = e = 0, f \neq 0$; but excludes the entire plane since we impose that $(a, b, c, d, e, f) \neq 0$.

A parabolic region $P_r \subset \mathbb{R}^2$ is formed by the intersection of a finite number of parabolic inequalities. It can be written as

$$P_r = \{x \in \mathbb{R}^2 : a_i x^2 + b_i xy + c_i y^2 + d_i x + e_i y + f_i \leq 0, i = 1, \dots, k\},$$

where $(a_i, b_i, c_i, d_i, e_i, f_i) \neq 0$, and $b_i^2 - 4a_i c_i = 0$. The following sets are special cases of parabolic regions: polyhedral sets, polyhedral cones, polytopes, edges, and vertices. The set $\{(x, y) \in \mathbb{R}^2 : y \geq x^2\}$ is an example of a non-polyhedral parabolic region while the set $\{(x, y) : y \leq x^2, y \geq 1\}$ is a nonconvex non-connected parabolic region.

A convex set $R \subset \mathbb{R}^2$ is called a parabolic subdivision if R can be written as the finite union of parabolic regions R_i , i.e., $R = \bigcup_{i=1}^n R_i$ and for any $j, k \in \{1, \dots, n\}$, $j \neq k$, the intersection $R_j \cap R_k$ is either empty or is contained in a parabola. A polyhedral subdivision is a special case of a parabolic subdivision (since lines are a special case of parabolas).

A face of a parabolic region $P_r \subset \mathbb{R}^2$ is defined as the interior of a nonempty intersection of a finite number of parabolic inequalities. It can be written as

$$P_r = \{x \in \mathbb{R}^2 : a_i x^2 + b_i xy + c_i y^2 + d_i x + e_i y + f_i < 0, i = 1, \dots, k\},$$

where $(a_i, b_i, c_i, d_i, e_i, f_i) \neq 0$, and $b_i^2 - 4a_i c_i = 0$.

Finally, we use standard definitions for subdifferentials, subgradients, and normal cones [38–41].

3 Computing conjugates

While the convex envelope of a piecewise function is not always the same as the convex envelope of each piece, for bivariate quadratic polynomials we can leverage the convex envelope of each piece to obtain the convex envelope. Let us denote the i^{th} piece of a piecewise function f by (f_i, P_i) where f_i is a function and P_i is a set. From [40, Theorem 3.4.1], we have $(\inf_i f_i)^* = \sup_i f_i^*$. Using [40, Proposition 2.6.1] and the biconjugate theorem $\text{conv } f = f^{**}$, we get

$$\begin{aligned} \text{conv}[\inf_i (f_i + I_{P_i})] &= \text{conv}[\inf_i (\text{conv}(f_i + I_{P_i}))], \\ &= [\inf_i (\text{conv}(f_i + I_{P_i}))]^{**}, \\ &= [\sup_i ([\text{conv}(f_i + I_{P_i})]^*)]^*. \end{aligned}$$

Taking the conjugate on both sides, we obtain

$$f^* = \left(\text{conv}[\inf_i (f_i + I_{P_i})] \right)^* = \sup_i ([\text{conv}(f_i + I_{P_i})]^*).$$

Hence we can find the conjugate of a bivariate PLQ function as follows.

1. Compute the convex envelope of each piece $\text{conv}(f_i + I_{P_i})$.
2. Compute the conjugate of each convex piece $[\text{conv}(f_i + I_{P_i})]^*$.
3. Compute the maximum of the conjugates over the entire PLQ function to obtain $f^* = \sup_i (\text{conv}(f_i + I_{P_i})^*)$.

Step 1 of this process, finding the convex envelope of each piece (f, P) with $f(x) = x^T A x + b^T x + c$, is computed as follow. We compute the eigenvalues of A . If they are both nonnegative, the function is convex and we are done. If they are both nonpositive, the function

is concave and the convex envelope is obtained as the convex hull of the points $\{(x, f(x)) : x \text{ is a vertex of } P\}$. If A is indefinite (one positive and one negative eigenvalue), we note that $\text{conv } f(x) = \text{conv}(x^T A x) + b^T x + c$; see [39, P. 93]. Hence, we only focus on the quadratic part. We rotate the polyhedral set so that in the new basis, the function is now $x \mapsto x_1 \cdot x_2$. We then use [5] where a method to compute the convex envelopes of bilinear forms over general polytopes is given.

Kumar [42] showed how to compute the conjugate of a rational function defined on a polytope, which is Step 2 of the above process.

Our focus is to explain how to perform Step 3, the last step required to obtain the conjugate, by computing the maximum of the functions obtained in Step 2 thereby obtaining the conjugate of a bivariate PLQ function. Repeating steps 2 and 3 to compute the biconjugate $f^{**} = (\sup_i (\text{conv}(f_i + I_{P_i})^*))^*$ is left for future work.

We point out a significant simplification of the structure of the conjugate. Locatelli [5] showed that the convex envelope of a quadratic over a polytope is a piecewise function defined on a polyhedral subdivision where each function is either linear, quadratic, or a rational function (ratio of quadratic over linear). It turns out that these rational functions have a specific structure. Kumar [42] subsequently showed that the conjugate of any of those functions is a piecewise function defined over a parabolic subdivision where the restriction to each piece is either linear, quadratic, or a fractional form

$$g_f(s_1, s_2) = \frac{\psi_1(s_1, s_2)}{\zeta_{00} \sqrt{\psi_{\frac{1}{2}}(s_1, s_2)}} + \psi_0(s_1, s_2),$$

where $\psi_1, \psi_0, \psi_{\frac{1}{2}}$ are linear functions in $s = (s_1, s_2) \in \mathbb{R}^2$ (justifications are in [34] and details in [43]). It turns out that the special structure of the rational functions considered prevent this general fractional form to occur.

Proposition 1 Assume $f(x) = x^T A x + b^T x + c + I_P$ with $P = \text{conv}\{x_1, x_2, x_3\}$ a triangular set in $\mathbb{R}^2$. Then its convex envelope $\text{conv } f = \min f_i + I_{P_i}$ is a piecewise function over a polyhedral subdivision where f_i is quadratic, linear, or a rational function while P is polyhedral. The conjugate of each piece of $\text{conv } f$ is a piecewise quadratic function defined over a parabolic subdivision.

Proof Locatelli [5] showed that $\text{conv } f$ is a piecewise function over a polyhedral subdivision that can be linear, quadratic, or rational. Kumar [34] showed that the conjugate of each piece is either a quadratic function, or a fractional function defined over a parabolic subdivision. We only need to prove that the fractional form never occurs.

The fractional form only appears in the conjugate when we compute the conjugate of a rational function. We only get a rational function as the convex envelope when we solve the optimization problem from Locatelli with $\eta_h = -(a + mb - q)^2/4m - bq$ and η_w linear. In this case η_h corresponds to the convex edge $y = mx + c$ and η_w corresponds to the vertex (x_1, y_1) . The optimization problem we are solving is

$$\max_{a,b} \{\eta_w - ax - by : \eta_w = \eta_h, (a, b) \in S_r\}$$

which gives a solution which is a rational function

$$r(x, y) = \frac{ax^2 + bxy + cy^2 + dx + ey + f}{gx + hy + k}$$

where $a = -my_1$, $b = q$, $c = x_1$, $d = -qy_1 + mx_1y_1$, $e = -qx_1 - x_1y_1$, $f = qx_1y_1$, $g = -m$, $h = 1$, and $k = -y_1mx_1$. (The rational function is extended by continuity at vertex (x_1, y_1) .) The computation is verified symbolically in Appendix [Appendix B.2](#) (file `vertexNan.m`).

From [34], the conjugate of the rational function of this form is a piecewise quadratic function defined on a parabolic division (a quadratic function is obtained for the conjugate corresponding to the convex edge while all other functions are linear). All other functions from the convex envelope, linear or quadratic, result in linear or quadratic functions for their conjugates. $\square$

Our implementation in MATLAB uses symbolic computation and rational numbers to avoid any floating-point errors. Using floating-point arithmetic can give rise to partitions of the domain with degenerate subsets, and floating-point coefficients of functions make it difficult to detect equal functions on adjacent domains. Indeed, one of the motivations of [31] is to check that intermediate computations of conjugates are still convex (as they should be without floating point errors). After observing these difficulties, we decided to work with symbolic representation of our functions with rational coefficients.

3.1 Algorithm

We use the symbols q for quadratic functions in the primal space, r for rational functions in the primal space, and s for quadratic functions in the dual space. We now fix our notations for each intermediate computation by naming each of the piecewise function involved.

The original PLQ function and its domain are noted f and $\text{dom } f$ where

$$f = \min_{i=1\dots n_q} (q_i + I_{\text{dom } q_i}),$$

where q_i is the quadratic function defining the i^{th} piece, and f has n_q pieces.

The convex envelope of $q_i + I_{\text{dom } q_i}$ is denoted

$$\text{conv}(q_i + I_{\text{dom } q_i}) = \min_{j=1\dots n_{r_i}} (r_{i,j} + I_{\text{dom } r_{i,j}}).$$

Here $r_{i,j}$ represents a rational function that defines the j^{th} piece of the convex envelope of the i^{th} piece of f .

The conjugate of $r_{i,j} + I_{\text{dom } r_{i,j}}$ is written

$$(r_{i,j} + I_{\text{dom } r_{i,j}})^* = \min_{k=1\dots n_{s_{i,j}}} (s_{i,j,k} + I_{\text{dom } s_{i,j,k}}),$$

where $s_{i,j,k}$ represents a quadratic function which is the k^{th} piece of $(r_{i,j} + I_{\text{dom } r_{i,j}})^*$.

The conjugate of $\text{conv}(q_i + I_{\text{dom } q_i})$ is

$$\begin{aligned} [\text{conv}(q_i + I_{\text{dom } q_i})]^* &= \max_{j=1\dots n_{r_i}} \min_{k=1\dots n_{s_{i,j}}} (s_{i,j,k} + I_{\text{dom } s_{i,j,k}}), \\ &= \min_{k=1\dots n_{s_i}} (s_{i,k} + I_{\text{dom } s_{i,k}}). \end{aligned}$$

The last equality defines our notation. The conjugate is a piecewise function with n_{s_i} pieces; each quadratic function is denoted $s_{i,k}$.

Finally, the conjugate of f is denoted

$$f^* = \max_{i=1\dots n_q} (s_{i,k} + I_{\text{dom } s_{i,k}}) = \min_{k=1\dots n_s} (s_k + I_{\text{dom } s_k}).$$

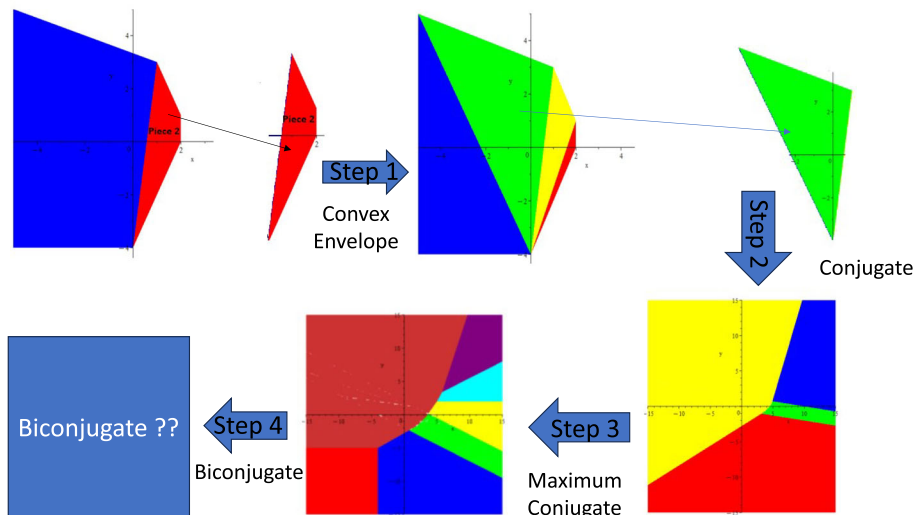


Fig. 1 Illustration of the steps taken on the domains to compute the biconjugate of a PLQ function. Note that the biconjugate has a polyhedral subdivision, but an unknown explicit formula leading us to label the last box “Biconjugate ??”

It is a piecewise function with n_s pieces where each quadratic function is denoted s_k .

While we use the same notation s , the number of indices indicate which function it is referring: 1 index for a piece of f^* , 2 indices for a piece of $[\text{conv}(q_i + I_{\text{dom } q_i})]^*$, and 3 indices for a piece of $(r_{i,j} + I_{\text{dom } r_{i,j}})^*$.

We illustrate the overall flow of computing the conjugate of the PLQ function in Figs. 1 and 2. We are given a PLQ function with $n_q = 2$ pieces. Each piece has a nonconvex bivariate function defined over a polyhedral region. As an example in Fig. 2, we have a PLQ function with two pieces.

The algorithm follows the following steps

1. Given q_i compute the convex envelope of $q_i + I_{\text{dom } q_i}$ to obtain $r_{i,j}$.
2. From $r_{i,j}$, compute the conjugate of $r_{i,j} + I_{\text{dom } r_{i,j}}$ to obtain $s_{i,j,k}$.
3. From $s_{i,j,k}$, compute the conjugate of $\text{conv}(q_i + I_{\text{dom } q_i})$ to obtain $s_{i,k}$, and deduce s_k to obtain the conjugate of f .

We apply Step 1 and get the convex envelope for each piece, as shown in Fig. 2b and c. This is followed by Step 2 to obtain the conjugate of each convex piece. The conjugates are shown in Fig. 2d–2g and finally the maximum of the conjugates are computed in Step 3. Fig. 2h, i show the conjugate of each piece and finally Fig. 2j shows the conjugate of the entire PLQ function.

From [5, Theorem 1.1], we know that the convex envelope of $q_i + I_{\text{dom } q_i}$ has a polyhedral subdivision where each piece is associated with a function of the form

$$\frac{\alpha_6 x^2 + \alpha_5 y^2 + \alpha_4 xy + \alpha_3 x + \alpha_2 y + \alpha_1}{\beta_3 x + \beta_2 y + \beta_1}.$$

We refer to [5] for the justification and to [43] for implementation details using our notations.

Step 2 computes the conjugate of each rational function over a polytope $r_{i,j} + I_{\text{dom } r_{i,j}}$ to obtain a piecewise function defined on a parabolic subdivision and one of the following

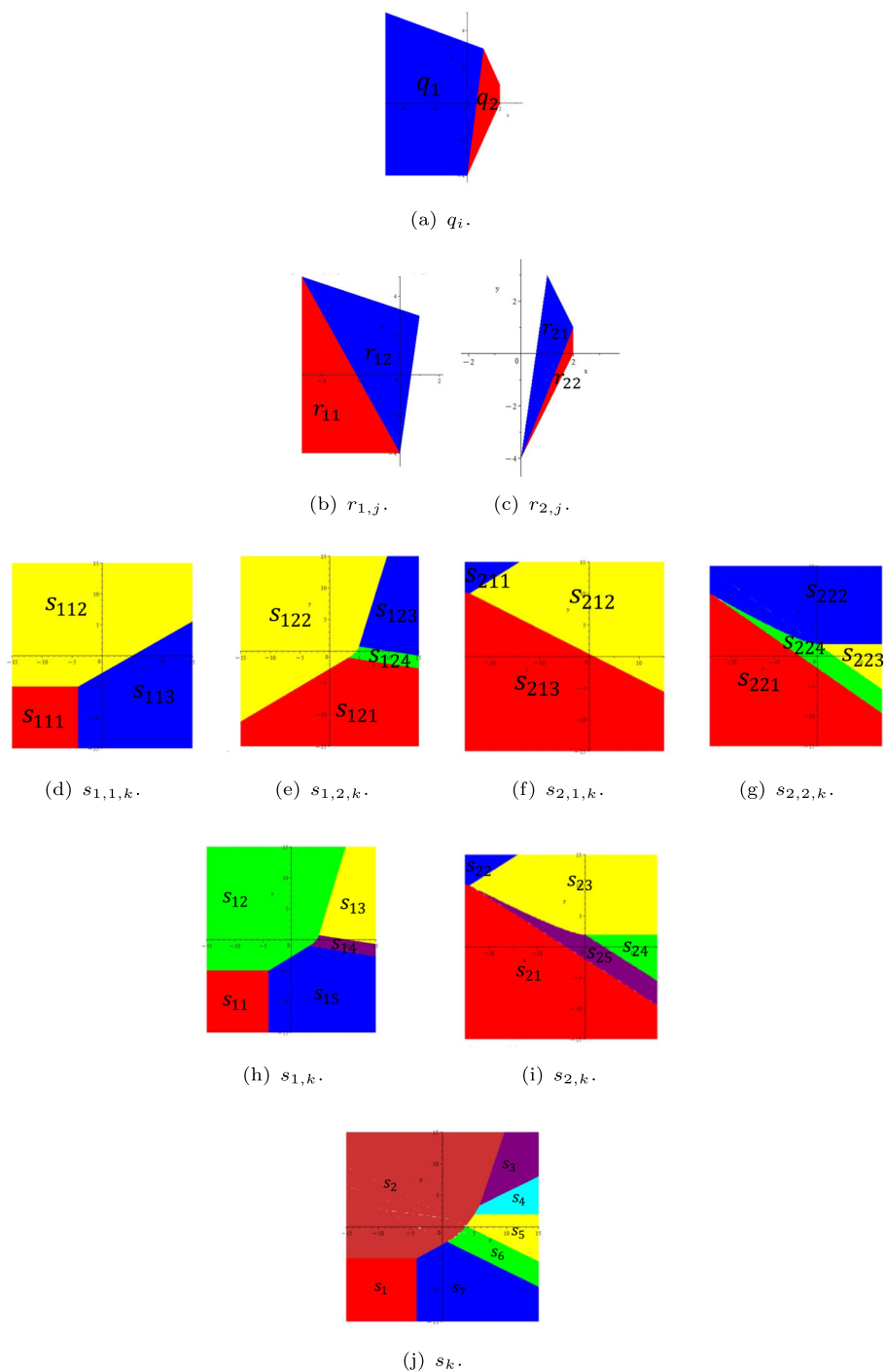


Fig. 2 Steps to compute the conjugate of a PLQ Function illustrated on the domain of the respective functions.(Functions defined in Table 1)

Table 1 Function definitions for Fig. 2

Function name	Expression
q_1	xy
q_2	xy
r_{11}	$-4x - 5y - 20$
r_{12}	$(155x - 5y + 4xy + 35x^2 + 5y^2 - 100)/(7x - y + 40)$
r_{21}	$-24x + 10y + 40$
r_{22}	$(8x + 6y - 4xy - 2x^2 + 2y^2 - 8)/(y - 2x + 3)$
s_{111}	$-5s_1 + 4s_2 - 20$
s_{112}	$-5s_1 + 5s_2 + 25$
s_{113}	$-4s_2$
s_{121}	$-4s_2$
s_{122}	$-5s_1 + 5s_2 + 25$
s_{123}	$s_1 + 3s_2 - 3$
s_{124}	$\frac{1}{28}s_1^2 + \frac{1}{2}s_1s_2 + \frac{2}{7}s_1 + \frac{7}{4}s_2^2 - 2s_2 + \frac{4}{7}$
s_{211}	$s_1 + 3s_2 - 46$
s_{212}	$2s_1 + s_2 - 2$
s_{213}	$-4s_2$
s_{221}	$-4s_2$
s_{222}	$2s_1 + s_2 - 2$
s_{223}	$2s_1$
s_{224}	$\frac{1}{8}s_1^2 + \frac{1}{2}s_1s_2 + s_1 + \frac{1}{8}1s_2^2 - 2s_2 + 2$
s_{11}	$-5s_1 + 4s_2 - 20$
s_{12}	$-5s_1 + 5s_2 + 25$
s_{13}	$s_1 + 3s_2 - 3$
s_{14}	$\frac{1}{28}s_1^2 + \frac{1}{2}s_1s_2 + \frac{2}{7}s_1 + \frac{7}{4}s_2^2 - 2s_2 + \frac{4}{7}$
s_{15}	$-4s_2$
s_{21}	$-4s_2$
s_{22}	$s_1 + 3s_2 - 46$
s_{23}	$2s_1 + s_2 - 2$
s_{24}	$2s_1$
s_{25}	$\frac{1}{8}s_1^2 + \frac{1}{2}s_1s_2 + s_1 + \frac{1}{8}1s_2^2 - 2s_2 + 2$
s_1	$-5s_1 + 4s_2 - 20$
s_2	$-5s_1 + 5s_2 + 25$
s_3	$s_1 + 3s_2 - 3$
s_4	$2s_1 + s_2 - 2$
s_5	$2s_1$
s_6	$\frac{1}{8}s_1^2 + \frac{1}{2}s_1s_2 + s_1 + \frac{1}{8}1s_2^2 - 2s_2 + 2$
s_7	$-4s_2$

functional forms

$$\begin{aligned} g_q(s_1, s_2) &= \zeta_{11}s_1^2 + \zeta_{12}s_1s_2 + \zeta_{22}s_2^2 + \zeta_{10}s_1 + \zeta_{01}s_2 + \zeta_{00}, \\ g_l(s_1, s_2) &= \zeta_{10}s_1 + \zeta_{01}s_2 + \zeta_{00}, \end{aligned} \quad (1)$$

where $\zeta_{ij} \in \mathbb{R}$. Justifications are in [34] and details in [43].

To complete Step 3 and obtain the conjugate of the entire PLQ function, we work in two stages: Step 3a given $s_{i,j,k}$, compute $s_{i,k}$, and Step 3b given $s_{i,k}$, compute s_k .

3.1.1 Step 3a

We divide the task into three steps: (i) compute intersection of the domains, (ii) compute maximum and if needed split the domain, and (iii) merge adjacent regions if they have the same conjugate expressions.

Step 3a(i) Let us first consider the domains of the conjugates corresponding to the first two convex functions in the i^{th} piece. The domain of the first conjugate is $\cup_{k_1=1 \dots n_{s_{i,1}}} \text{dom } s_{i,1,k_1}$ while $\cup_{k_2=1 \dots n_{s_{i,2}}} \text{dom } s_{i,2,k_2}$ is the domain of the second conjugate of the i^{th} piece. We need to find the intersection $\text{dom } s_{i,k}^t = \text{dom } s_{i,1,k_1} \cap \text{dom } s_{i,2,k_2}$, for all k_1, k_2 .

A simple algorithm is to exhaustively generate all pairs between the two sets of regions and check if the intersection of the interior of the two regions is nonempty. When we find a nonempty intersection, we store it in the output list along with the two functions which were defined on the two regions we were intersecting. So we are basically storing nonempty intersections along with two functions, as $(\text{dom } s_{i,k}^t, [s_{i,k,1}, s_{i,k,2}]) = (\text{dom } s_{i,1,k_1} \cap \text{dom } s_{i,2,k_2}, [s_{i,1,k_1}, s_{i,2,k_2}])$. The output $\text{dom } s_{i,k}^t$ is a list of nonempty domains covering the entire $\mathbb{R}^2$ plane as shown in Fig. 3.

The above algorithm considers all pairs so it has a complexity of $O(n^2)$, where $n = \max(n_{s_{i,1}}, n_{s_{i,2}})$, but it is possible to reduce it to $O(n)$. When we consider regions that are neighbors, we get nonempty intersections only when the original convex envelopes have a common vertex or edge. So a propagation algorithm similar to [22] provides a linear number of nonempty intersections. Computing the intersection of two convex polytopes is linear in the number of vertexes [44] resulting in a linear-time algorithm. Practically, starting with a triangulation, we have a bounded number of vertices. Computing a triangulation for a convex polyhedral set takes linear time since triangulating such sets only require joining non-neighbor vertices from any arbitrary vertex [45, p. 49]. Hence, our approach runs in linear-time.

At this stage we are finding the intersection of domains of the original conjugates. These domains are parabolic subdivisions [34]. After computing the intersection we again get a parabolic subdivision.

Proposition 2 *The output of Step 3a(i) is a parabolic subdivision.*

Proof Each $\text{dom } s_{i,j,k}$ is a parabolic subdivision, which is the nonempty interior of a set of parabolic inequalities. So each domain is of the form $\text{dom } s_{i,j,k} = \{(x, y), a_l x^2 + b_l x y + c_l y^2 + d_l x + e_l y + f_l < 0, l = 1 \dots K\}$ where $b_l^2 - 4a_l c_l = 0$, and $K \in \mathbb{N}$ is the number of inequalities defining $\text{dom } s_{i,j,k}$. When we find the intersection of two such domains, we only store the regions with nonempty interiors. These are again defined by parabolic inequalities of the same form, hence the subdivision is still parabolic. $\square$

We illustrate Step 3a with an example where V denotes a vertex and E denotes the relative interior of an edge. Fig. 4a, shows the convex polyhedral subdivision of the domain

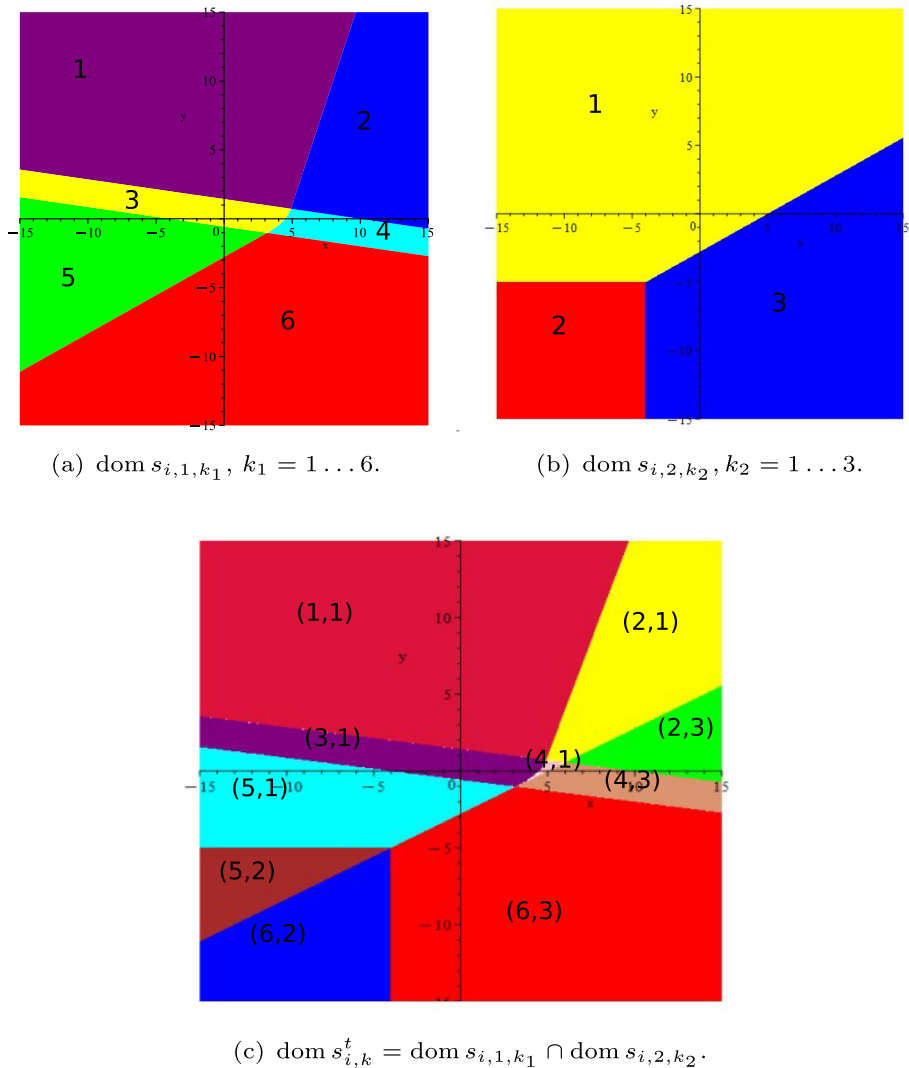


Fig. 3 Intersection of domains of two conjugates

of one piece as obtained in Step 1. As a result of Step 1 we get $r_{i1} = \text{conv}(V_1, V_2, V_3)$ and $r_{i2} = \text{conv}(V_2, V_3, V_4)$. Fig. 4b and c show $\text{dom}(s_{i1})$ and $\text{dom}(s_{i2})$ respectively. From Fig. 4a we observe that $\text{dom}(r_{i1}) \cap \text{dom}(r_{i2}) = \{V_2, V_3, E_{23}\}$. The set $\text{dom}(s_{i1})$ has pieces corresponding to $R_1 = \{V_1, V_2, V_3\}$, whereas $\text{dom}(s_{i2})$ has pieces corresponding to $R_2 = \{V_2, V_3, V_4, E_{23}, E_{24}, E_{34}\}$. In Fig. 4d, the nonempty intersections always have an entity corresponding to $R_1 \cap R_2$.

Step 3a(ii) From Step 3a(i) we get a list $\text{dom } s_{i,k}^t$, along with two functions $s_{i,k,1}$ and $s_{i,k,2}$ defined over each $\text{dom } s_{i,k}^t$. We now find the maximum function $s_{i,k}^t = \max\{s_{i,k,1}, s_{i,k,2}\}$ over the given $\text{dom } s_{i,k}^t$. In doing so it is possible that $\text{dom } s_{i,k}^t$ is further divided into two

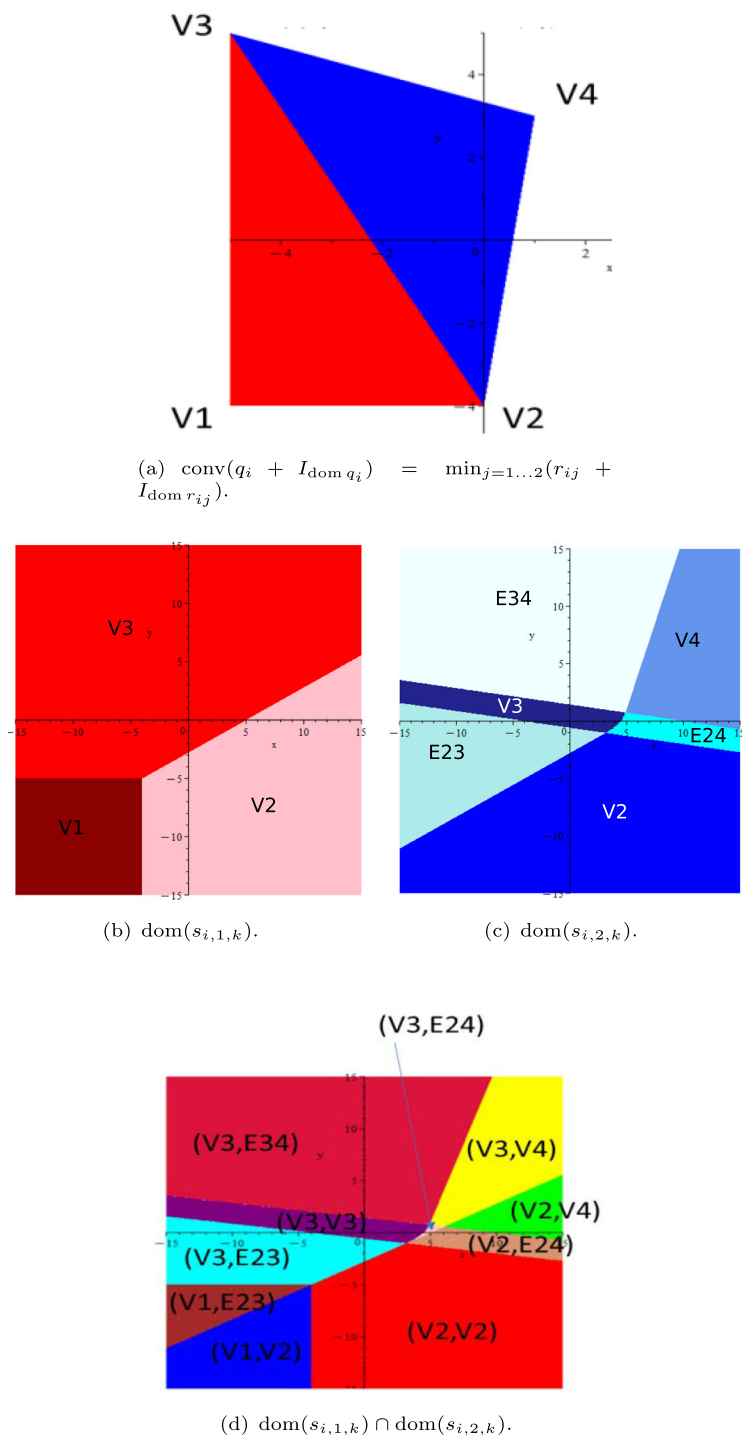


Fig. 4 Computing $\text{dom}(s_{i,1,k}) \cap \text{dom}(s_{i,2,k})$ using $\text{dom}(r_{i,1}) \cap \text{dom}(r_{i,2})$

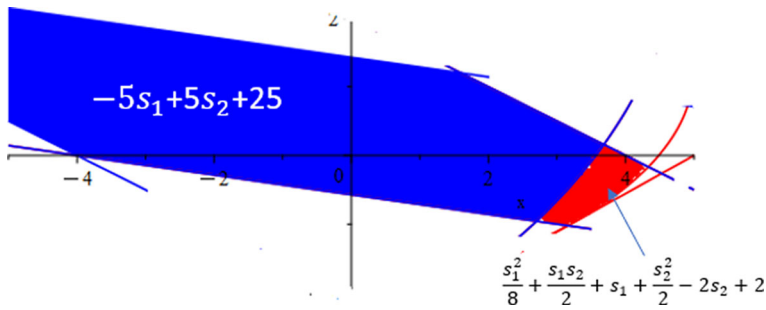


Fig. 5 Further division by a parabolic curve as explained in Example 4

regions one where $s_{i,k,1}$ is the maximum, and the other where the maximum is $s_{i,k,2}$; or we get the same region with either $s_{i,k,1}$ or $s_{i,k,2}$ as the maximum.

Example 4 The blue region in Fig. 5 is denoted by B while R is the red region; they are defined as

$$B = \{(u, v) : -u - 7v - 4 \leq 0, u + 7v - 10 \leq 0, -u - 2v - 4 \leq 0, \\ u + 2v - 4 \leq 0, 48u - 56v + 4uv + u^2 + 4v^2 - 184 \leq 0\};$$

$$R = \{(u, v) : -u - 7v - 4 \leq 0, 148u - 196v + (u + 7v)^2 - 684 \leq 0, u + 2v - 4 \leq 0, \\ 56v - 48u - 4uv - u^2 - 4v^2 + 184 \leq 0\}.$$

Functions $f_1(u, v) = -5u + 5v + 25$ and $f_2(u, v) = u^2/8 + uv/2 + u + v^2/2 - 2v + 2$ are defined on $B \cup R$. Computing $\max\{f_1, f_2\}$ over $B \cup R$ gives a further subdivision along $f_1 - f_2 = 0$; see Fig. 5. Function f_1 is the maximum over B while f_2 is over R .

At this point, when we find the maximum of the two functions defined on the intersecting domain, we get a further subdivision with an inequality $f_1 - f_2 \leq 0$ for one region and $f_2 - f_1 \leq 0$ for the other. Thus in the maximum we can get a subdivision which is the difference of the two conjugate functions, f_1 and f_2 . From [34, Theorem 4.24, 4.27] we get the conjugate functions of the forms (1). In order to find the possible subdivisions, we enumerate all the pairs, of the conjugate functions, (f_1, f_2) that give us a further subdivision of the domain. We first give propositions for each possible case and the partition it can give us and then put everything together in Theorem 6.

Proposition 3 *Let the two functions defined on the parabolic domain be linear. If we find the $\max\{f_1, f_2\}$, we can get a further subdivision of the domain. This division is given by a linear inequality hence the subdivision remains parabolic.*

Proof The domain is split by the equality, $f_1 - f_2 = 0$ which is linear. $\square$

In order to get the subdivision when one conjugate expression is quadratic, we first need to show that the quadratic expression g_q obtained in (1) is always parabolic.

Proposition 4 *The quadratic expression g_q obtained in (1) is parabolic.*

Proof The conjugate expression obtained is a quadratic only in a certain condition. In Step 1, when we find the convex envelope of each piece, we can obtain a bivariate rational function, $\frac{\xi_1^2(x)}{\xi_2(x)} + \xi_0(x)$, where $\xi_0(x) = \xi_{01}x_1 + \xi_{02}x_2 + \xi_{00}$, $\xi_1(x) = \xi_{11}x_1 + \xi_{12}x_2 + \xi_{10}$ and

$\xi_2(x) = \xi_{21}x_1 + \xi_{22}x_2 + \xi_{20}$. Then in Step 2, we find the conjugate expression of this rational function. Using the method given in [34], when we find the expression on an edge ($y = mx + q$) of the polytope and $\xi_{21} + m\xi_{22} = 0$, we get a quadratic expression. The quadratic expression obtained is

$$f^*(s) = \zeta_{11}s_1^2 + \zeta_{12}s_1s_2 + \zeta_{22}s_2^2 + \zeta_{10}s_1 + \zeta_{01}s_2 + \zeta_{00}, \quad (2)$$

where

$$\begin{aligned} \zeta_{11} &= -\frac{(\xi_{11}\gamma_{10} + m\xi_{12}\gamma_{10})^2}{\xi_{20} + q\xi_{22}} + \gamma_{10}, \\ \zeta_{12} &= -\frac{2(\xi_{11}\gamma_{01} + m\xi_{12}\gamma_{01})(\xi_{11}\gamma_{10} + m\xi_{12}\gamma_{10})}{\xi_{20} + q\xi_{22}} + \gamma_{01} + m\gamma_{10}, \\ \zeta_{22} &= -\frac{(\xi_{11}\gamma_{10} + m\xi_{12}\gamma_{10})^2}{\xi_{20} + q\xi_{22}} + \gamma_{10}m, \\ \zeta_{10} &= -\frac{2(\xi_{11}\gamma_{10} + m\xi_{12}\gamma_{10})(\xi_{10} + \xi_{11}\gamma_{00} + \xi_{12}(q + m\xi_{00}))}{\xi_{20} + q\xi_{22}} + \\ &\quad \gamma_{00} - m\xi_{02}\gamma_{10} - \xi_{01}\gamma_{10}, \end{aligned} \quad (3)$$

$$\begin{aligned} \zeta_{01} &= -\frac{2(\xi_{11}\gamma_{01} + m\xi_{12}\gamma_{01})(\xi_{10} + \xi_{11}\gamma_{00} + \xi_{12}(q + m\xi_{00}))}{\xi_{20} + q\xi_{22}} + \\ &\quad m\gamma_{00} - m\xi_{02}\gamma_{01} - \xi_{01}\gamma_{01} + q, \end{aligned} \quad (4)$$

and

$$\zeta_{00} = -\frac{(\xi_{10} + \xi_{11}\gamma_{00} + \xi_{12}(q + m\xi_{00}))^2}{\xi_{20} + q\xi_{22}} - \xi_{00} - \xi_{01}\gamma_{00} - \xi_{02}(m\gamma_{00} + q).$$

Now substituting the above values, we obtain

$$\zeta_{12}^2 - 4\zeta_{11}\zeta_{22} = 0.$$

Hence (2) is parabolic. $\square$

The MATLAB code to verify this proof is given in Section [Appendix B.1](#).

Lemma 5 *If f_1 is a parabolic quadratic function and f_2 is linear, computing $\max\{f_1, f_2\}$ may result in a division by the curve $f_1 - f_2 = 0$. In that case, the curve $f_1 - f_2 = 0$ is parabolic.*

Proof Let the parabolic expression defined be $f^1(s) = \zeta_{11}s_1^2 + \zeta_{12}s_1s_2 + \zeta_{22}s_2^2 + \zeta_{10}s_1 + \zeta_{01}s_2 + \zeta_{00}$ and the linear function be $f^2(s) = \zeta_{10}^1s_1 + \zeta_{01}^1s_2 + \zeta_{00}^1$. The new subdivision is given by $f^1(s) - f^2(s) = \zeta_{11}s_1^2 + \zeta_{12}s_1s_2 + \zeta_{22}s_2^2 + (\zeta_{10} - \zeta_{10}^1)s_1 + (\zeta_{01} - \zeta_{01}^1)s_2 + (\zeta_{00} - \zeta_{00}^1) = 0$. As $f^1(s)$ is parabolic, we have $\zeta_{12}^2 - 4\zeta_{11}\zeta_{22} = 0$, hence $f^1(s) - f^2(s)$ is also parabolic. $\square$

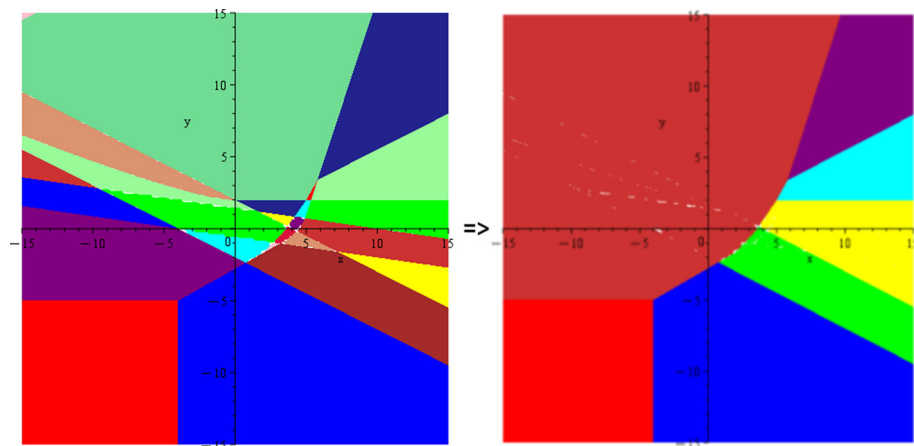
This is shown in Fig. 5.

Theorem 6 *The maximum of two conjugates is a piecewise function that admits a parabolic subdivision.*

Proof When we find the convex envelope of each piece in Step 1, we do not get rational functions in adjacent domains. They only arise when the region is divided into three regions. The middle region has the rational function, while the adjacent regions have linear expressions.

Table 2 Possible divisions of domain obtained in Step 3a (ii)

$s_{i,1,k}$	$s_{i,2,k}$	$s_{i,1,k} - s_{i,2,k}$
Linear	Linear	Linear
Linear	Parabolic	Parabolic

**Fig. 6** Merging adjacent pieces with the same function

In Step 2, only the rational functions restricted to an edge give rise to quadratic expressions. In Step 3, when we compare two functions we always get one of them as linear. Nonempty intersections are obtained only when the primal has a common edge. Hence the two functions we compare always have one linear function. Thus the additional subdivision is one of the forms given in Table 2. The proofs that these subdivisions are parabolic are given in propositions 3, and 5. $\square$

Step 3a(iii) At this point we computed the maxima and obtained the conjugate as a piecewise function. This can be further simplified by merging adjacent regions where functions are the same as shown in Fig. 6. Coefficients of the functions involved are stored as rational numbers, so we can determine two identical functions without floating point considerations.

For every piece of the PLQ function, we have multiple conjugates, r_{ijk} . So far, we have given a method to compute the maximum of two conjugates. We can now add one conjugate at a time to get the conjugate of the entire piece. We first copy the first conjugate into the output. We then iterate over the remaining pieces, performing steps 3a(i) and 3a(ii) and finally get the conjugate of the entire piece.

Division

Proposition 7 *The function $\max_j(r_{ijk})$ admits a parabolic subdivision.*

Proof By Proposition 2 and Theorem 6, the maximum of two conjugates is a parabolic subdivision. In computing the convex envelope of a piece only one division can have a rational function defined on it. In computing the conjugate, only the rational function would give us a quadratic form. Thus the maximum of multiple divisions is a parabolic subdivision. $\square$

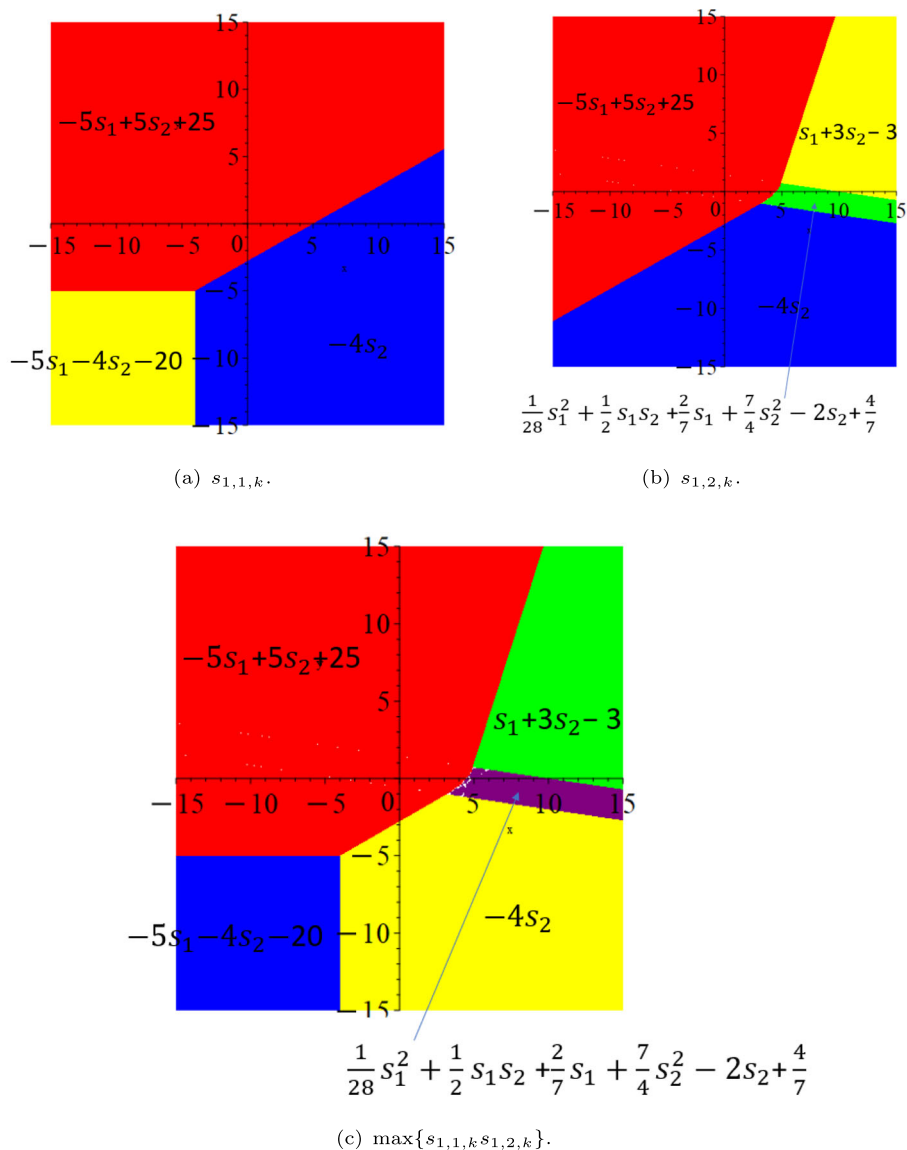


Fig. 7 Maximum Conjugate for piece 1

Example 5 In the example in Fig. 2(a), we had two pieces. On finding the convex envelope in Step 1 we obtained convex functions defined over a polyhedral subdivision, refer to Fig. 2(b-c). Corresponding to these, in Step 2 we got two piecewise conjugates as shown in Fig. 2(d-g). The maximum of the conjugates for Piece 1, is shown in Fig. 7.

The same process is repeated for Piece 2 shown in Fig. 1, and the maximum conjugate for this piece is shown in Fig. 8.

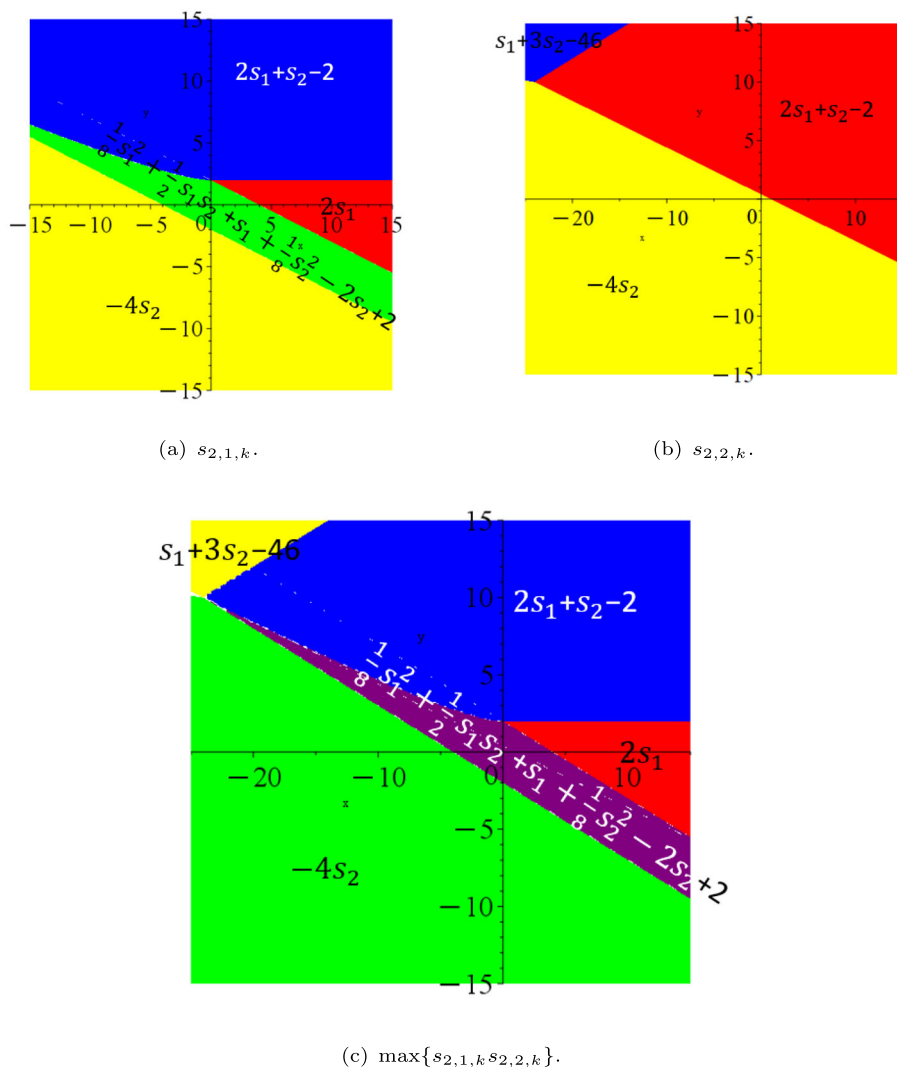


Fig. 8 Maximum Conjugate for piece 2

3.1.2 Step 3b

In Step 3a, corresponding to each piece of the PLQ function, we obtained a maximum conjugate which is the maximum of all the conjugates for that piece. In this stage we find the maximum over all these maximums to get one maximum for the entire PLQ function. The steps followed to compute this are very similar to those followed in Step 3a. In Step 3b(i) we find the intersection of the domains, $\text{dom } s_k^t = \text{dom } s_{1,k} \cap \text{dom } s_{2,k}$ and in Step 3b(ii), we find the maximum of two functions over the same domain as $s_k^t = \max\{s_{k,1}, s_{k,2}\}$ over $\text{dom } s_k^t$ and then merge to get the maximum over the first two pieces. We then iterate Step 3b(i) and Step 3b(ii), over the remaining pieces, $\text{dom } s_k^t = \text{dom } s_k^t \cap \text{dom } s_{j,k}$, $s_k^t = \max\{s_k^t, s_{j,k}\}$ and

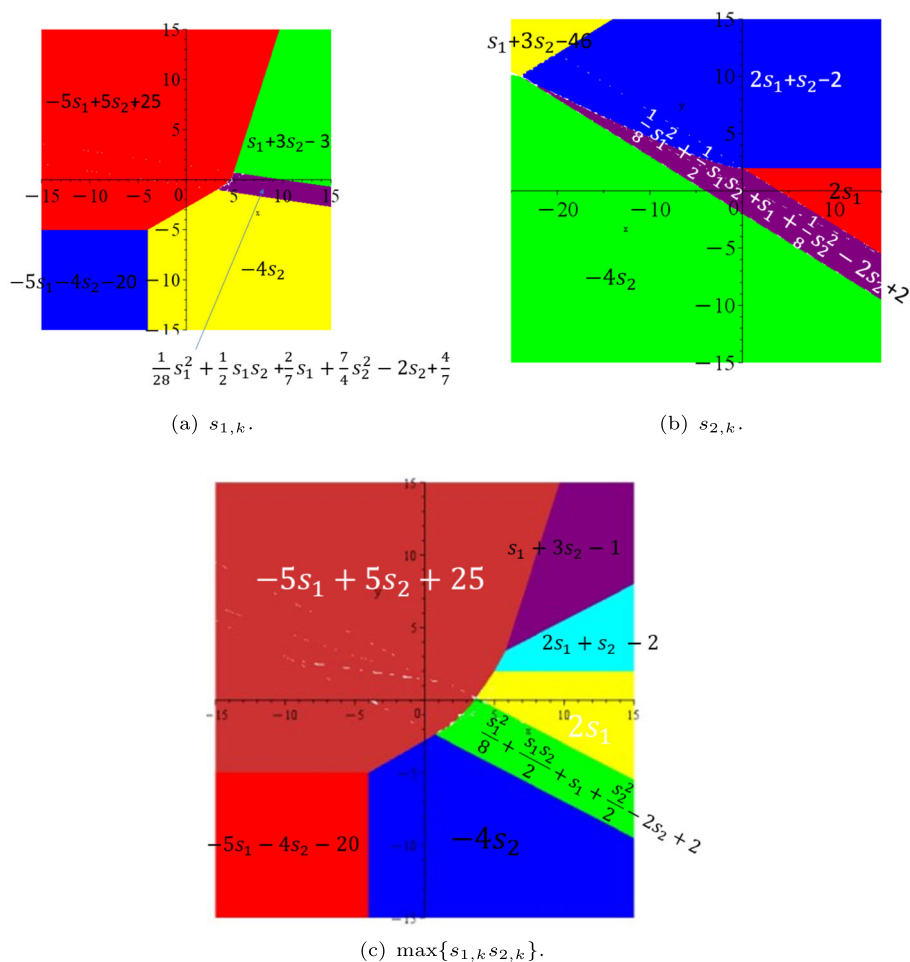


Fig. 9 Maximum Conjugate over all pieces

merge to finally get the maximum conjugate $s_k = s_k^t$, which is the conjugate of the PLQ function.

Divisions In this step we are finding the maximum between two pieces. In Step 3b(i), we find the intersection of parabolic subdivisions and hence get parabolic subdivisions as given in Proposition 2. In Step 3b(ii), we are finding the maximum of two functions over a parabolic domain. The pieces are adjacent to each other in the primal and Proposition 7 shows that we would get a parabolic subdivision as if we had considered both pieces together as one piece.

Example 6 Again, consider the example given in Fig. 1. We had two pieces. Corresponding to each piece, in Step 3a we got the maximum conjugate of each piece which were shown in Figs. 7 and 8. We now find the maximum over these maximums to get the conjugate of the PLQ function as shown in Fig. 9a, b.

Table 3 Function definitions for Fig. 10

Function name	Expression
q_1	xy
r_{11}	$-4x - 5y - 20$
r_{12}	$(30x - 5y + 4xy + 10x^2 + 5y^2 - 100)/(2 * x - y + 15)$
r_{13}	$5x + 2y - 10$
r_{14}	$\frac{29x}{5} + \frac{17y}{5} - 13$
s_{111}	$-5s_1 + 4s_2 - 20$
s_{112}	$-5s_1 + 5s_2 + 25$
s_{113}	$-4s_2$
s_{121}	$-4s_2$
s_{122}	$-5s_1 + 5s_2 + 25$
s_{123}	$2s_1$
s_{124}	$\frac{1}{8}s_1^2 + \frac{1}{2}s_1s_2 + s_1 + \frac{1}{8}s_2^2 - 2s_2 + 2$
s_{131}	$-5s_1 + 5s_2 + 25$
s_{132}	$s_1 + 3s_2 - 3$
s_{133}	$2s_1$
s_{141}	$-5s_1 + 5s_2 + 25$
s_{142}	$s_1 + 3s_2 - 3$
s_{143}	$2s_1 + s_2 - 2$
s_1	$-5s_1 + 4s_2 - 20$
s_2	$-5s_1 + 5s_2 + 25$
s_3	$s_1 + 3s_2 - 3$
s_4	$2s_1 + s_2 - 2$
s_5	$2s_1$
s_6	$\frac{1}{8}s_1^2 + \frac{1}{2}s_1s_2 + s_1 + \frac{1}{8}s_2^2 - 2s_2 + 2$
s_7	$-4s_2$

Example 7 Here we refer to the PLQ function shown in Fig. 1 but instead of dividing it into two pieces we take just one piece as illustrated in Fig. 10 and use this to test that the final output for both examples is the same. In this example as there is only one piece, Step 3b is not required.

Observing the conjugates we computed in Figs. 10, 11 and 12, we can estimate the number of pieces in the conjugate.

Conjecture 1 The number of pieces in the conjugate (computed as a maximum) is equal to the number of vertices plus the number of convex edges of the overall domain.

4 Examples

We performed a lot of experiments to test and time the code. The timings are recorded on a Intel(R) Xeon(R) CPU E5-1620 v4 @ 3.50GHz 3.50 GHz machine with 64.0 GB RAM, We have run the code with MATLAB R2023B on a Windows 10 operating system. The emphasis is on correct code, not speed so the code is not optimized.

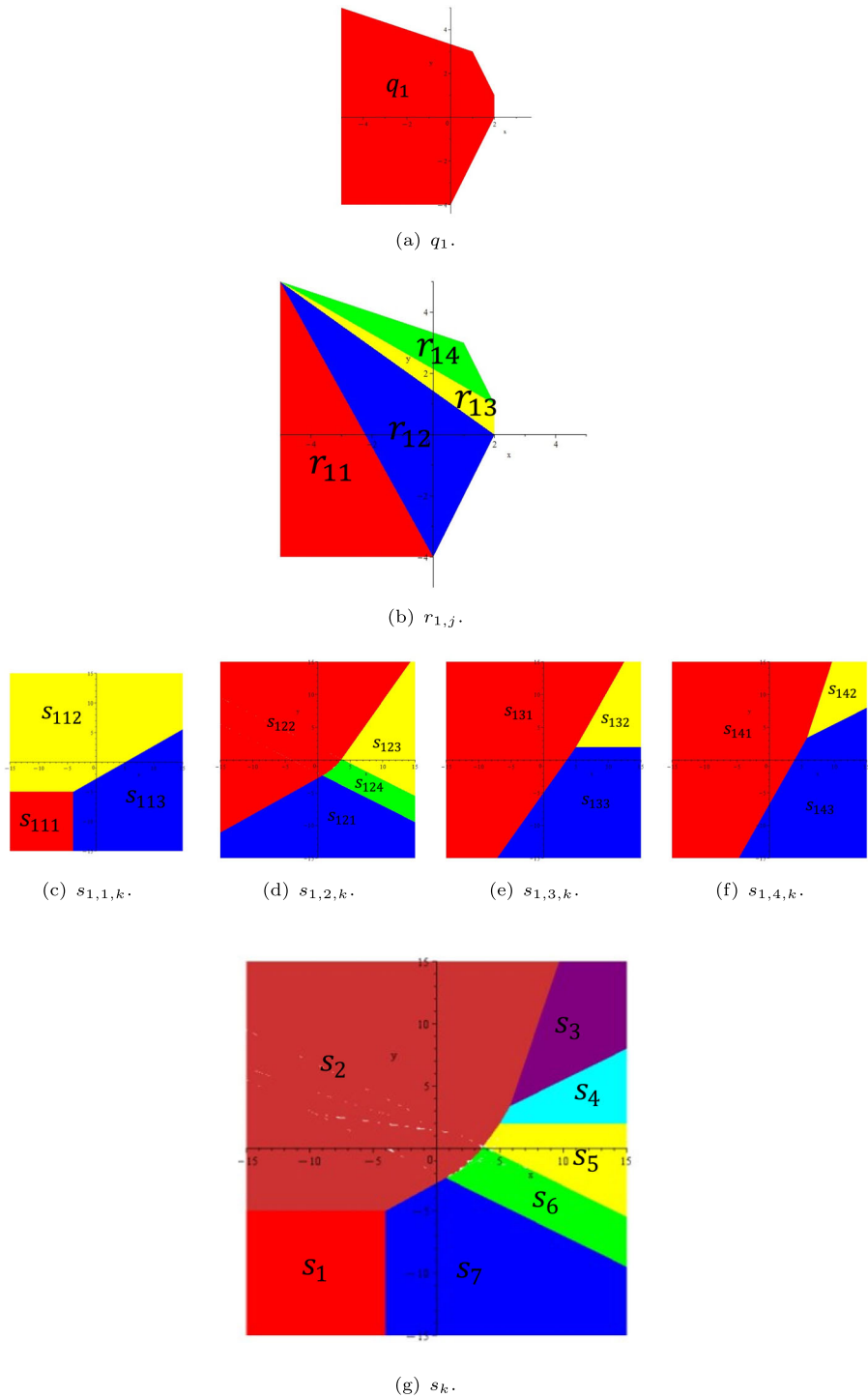


Fig. 10 Conjugate of Example 7. (The functions are enumerated in the Table 3)

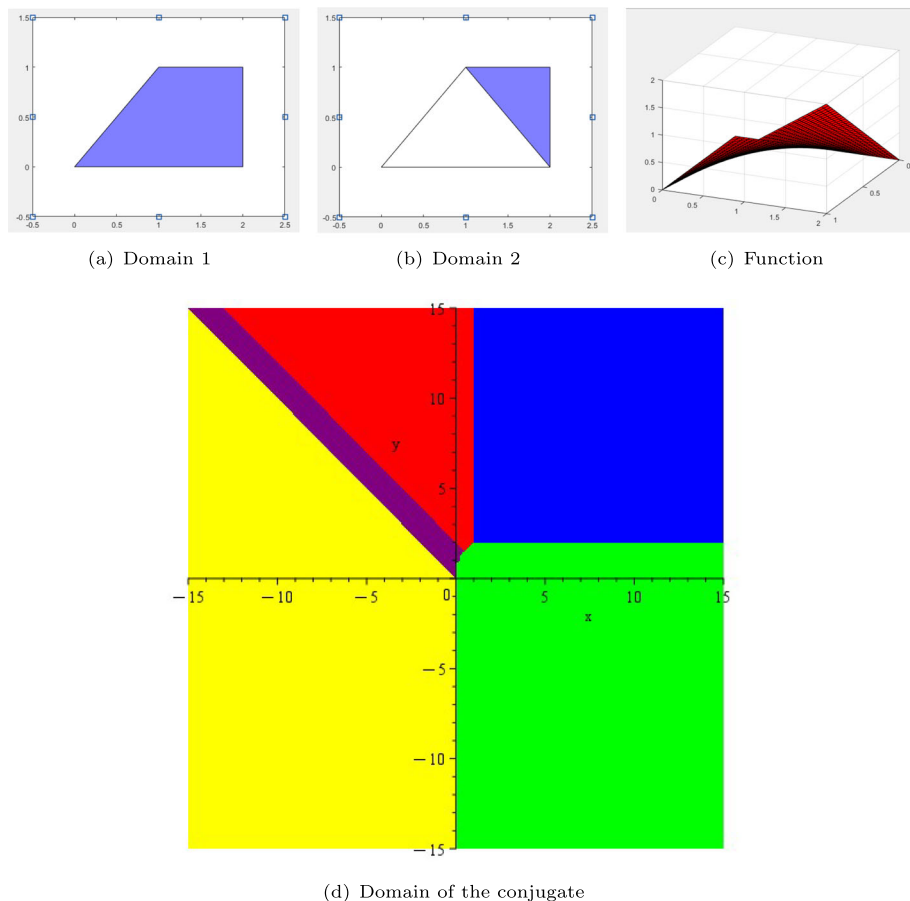


Fig. 11 Experiment 1: Computing the conjugate of a function with the domain stored as one piece gives the same result as the same function with the domain stored as two pieces

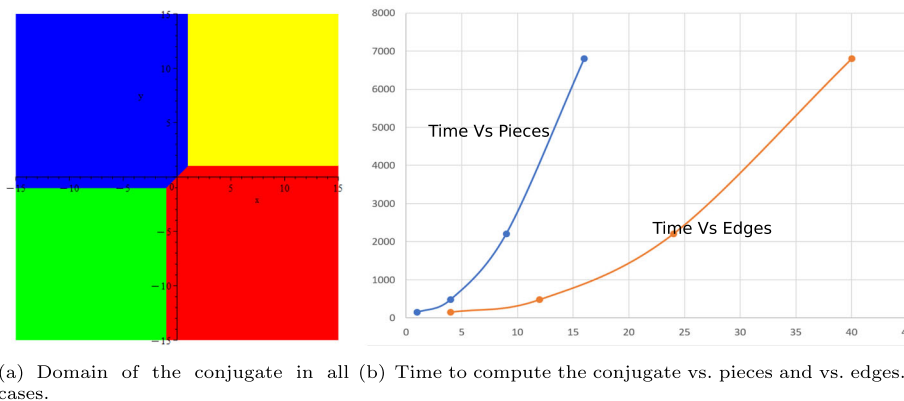


Fig. 12 Conjugate of xy over unit square with each piece subdivided into n^2 pieces and corresponding computation time per piece and per edge

4.1 Verification

In order to verify the code works correctly, we first verified small problems by solving them by hand. Next we computed conjugates for examples from [34] and verified that we obtain the same results. Finally we divided the same domain in different ways and check that we obtained the same conjugate (in the first two steps, the convex envelopes and conjugates computed are different). In all cases, we found that Step 3 computation is correctly implemented.

4.1.1 Experiment 1

We consider $f(x, y) = xy + I_{\text{conv}\{V_1, V_2, V_3, V_4\}}$ where $V_1 = (0, 0)$, $V_2 = (1, 1)$, $V_3 = (2, 1)$ and $V_4 = (2, 0)$. This is the example given in [42, Example 3.3].

This function and the domain of its conjugate are shown in Fig. 11. We repeat this experiment twice, once with one piece, second time with two pieces and obtain the same conjugate.

4.1.2 Experiment 2

We divide the same domain given in Fig. 2 in three different ways:

1. The first example has two pieces and is illustrated in Fig. 13(a).
2. The second example has one pieces and is illustrated in Fig. 13(b).
3. The third example again has two pieces and is illustrated in Fig. 13(c). Polyhedral division for this example is shown in Fig. 14.

Although the division is different, all three examples give the same conjugate as illustrated in Example 7.

4.1.3 Timing Tests

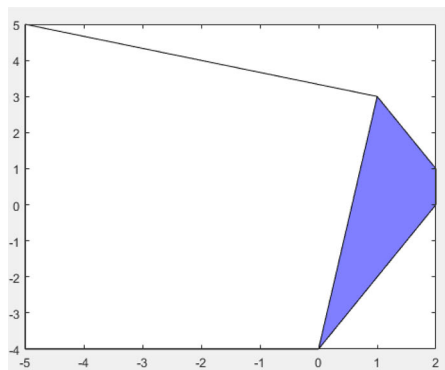
We find the conjugate of the function $f(x, y) = xy + I_S(x, y)$ where $S = \text{conv}\{V_1, V_2, V_3, V_4\}$ with $V_1 = (-1, -1)$, $V_2 = (-1, 1)$, $V_3 = (1, 1)$ and $V_4 = (-1, 1)$. We then divide this piece into n^2 pieces and repeat the experiment where $n = 1, 2, 3, 4$ and tabulate the results. All three tests give the same output which is given in Fig. 12(a) and the graph of the time to find conjugate vs. number of pieces and conjugate vs edges is plotted in Fig. 12(b).

The results for $n = 1$, $n = 2$ and $n = 3$ are tabulated in Tables 4, 5 and 6. The tables contain the computation time for each step for each piece, and the average time per piece, which allow to estimate the improvement we could get by parallelisation of the code. The results are summarized in Table 7. We plot graphs for time against the number of pieces originally given and the number of edges of the original domain. Both these graphs appear quadratic.

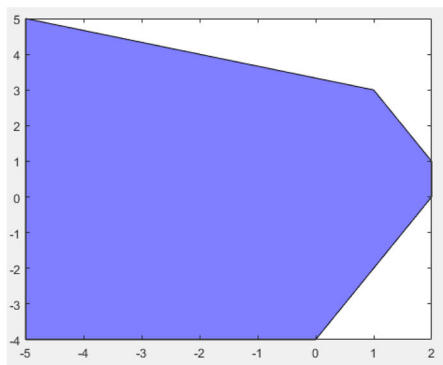
Using a better algorithm, we can reduce the computation time to linear.

Conjecture 2 Computing the conjugate of a PLQ function is linear with respect to the number of edges.

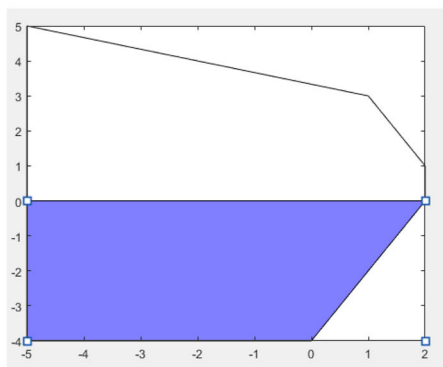
Timings for both Step 1 and Step 3 can be improved significantly. In Step 1, a lot of sub-problems can be discarded in advance. This code has not been implemented. In Step 3, we compute the Cartesian product of two list while computing the intersection of domains. The time taken for these loops can be reduced as this code can be implemented in linear time. Merging adjacent regions where possible would also decrease list sizes, thus giving us a faster time.



(a) Domain 1



(b) Domain 2



(c) Domain 3

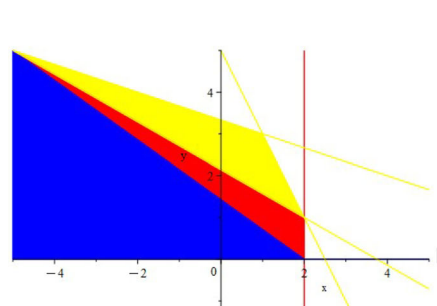
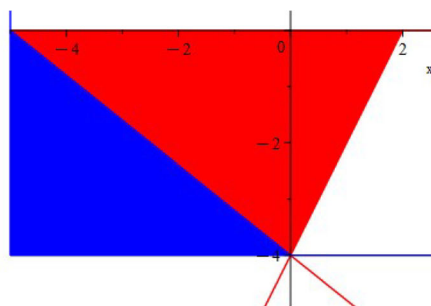
Fig. 13 Domains used in Experiment 2(a) r_1 .(b) r_2 .**Fig. 14** Polyhedral subdivision of domain of convex envelope for Fig. 13(c)

Table 4 Computation time to calculate the conjugate of $f(x, y) = xy + I_S(x, y)$ with $n = 1$ piece

Computation step	Time (s)
convex pieces	2
Time for Step 1	60.3
Time for Step 2	3.6
Time for Step 3a	9.1
Time for Step 3b	0.0
Total	73.1

Table 5 Computation time in seconds to calculate the conjugate of $f(x, y) = xy + I_S(x, y)$ with $n = 4$ pieces; each piece is split in 2 convex pieces

Step	Piece No				Total	Avg
	1	2	3	4		
Step 1	57.9	66.5	66.2	55.2	245.8	61.5
Step 2	5.3	4.9	4.8	2.9	17.9	4.5
Step 3a	12.6	12.7	12.9	8.0	46.2	11.6
Average per piece						77.5
Step 3b					168.4	
Total					478.3	

Table 6 Computation time in seconds to calculate the conjugate of $f(x, y) = xy + I_S(x, y)$ with $n = 9$ piece; each piece is split in 2 convex pieces

Step	Piece No									Total	Avg
	1	2	3	4	5	6	7	8	9		
Step 1	48.4	46.8	48.6	49.6	47.8	46.9	46.9	47.7	49.1	431.8	47.9
Step 2	3.4	3.7	3.6	3.5	3.6	3.9	3.7	3.5	3.6	32.4	3.6
Step 3a	9.1	9.9	3.6	9.1	8.9	11.2	9.2	9.1	9.6	79.5	8.8
Average											60.4
Step 3b										1,661.4	
Total										2,205.1	

Table 7 Timing summary in seconds

nPieces	nVertices	nEdges	Avg time for 1 piece	Total time
1	4	4	73	146
4	9	12	77	478
9	16	32	60	2,205
16	25	40	58	6,803

5 Conclusion

We have given a method to find the conjugate of a bivariate piecewise linear-quadratic function. We have implemented the first three steps of this method to get the conjugate of a bivariate PLQ function. It is possible to extend the code to implement Step 4 to find the biconjugate in order to get the convex envelope of the PLQ function. In the current implementation, the complexity of Step 1 can be exponential in the worst-case, Step 2 is linear and Step 3 is polynomial. But we expect the complexity to be reduced in the future.

5.1 Future work

The current implementation is in MATLAB. The code is written in a way that could be ported to other languages and architectures. It can be extended to incorporate the functionality that has been mentioned as future possibilities.

Future work can be pursued in several directions. Now that we have the conjugate of the PLQ function, the next natural step is to compute the biconjugate in order to obtain the convex envelope; this requires computing the conjugate of piecewise quadratic functions defined on parabolic subdivisions.

As we have implemented this computation in four steps, we can improve each of these steps. In Step 1, we can reduce the complexity of the algorithm. When there is no convex edge - complexity is $\binom{n}{2}$. When there is a convex edge, with a vertex opposite it - this gives a subdivision which is generally the maximum on solving the subproblems. Thus when there is a convex edge, we can reduce the number of subproblems being solved.

Step 3a can also be improved and implemented with a lower complexity. We have given an algorithm to implement the intersection of two domains in $O(n)$. It has also been observed that the conjugates corresponding to vertices and edges that are not a part of the division when we find the convex envelope of each piece appear exactly the same in the final conjugate. This fact can be used to develop a better algorithm to find the maximum. If you observe Fig. 15(a), the conjugates corresponding to the vertices (V_2, V_6) as illustrated in Fig. 15(b-c), which are not a part of the subdivision, appear exactly the same in the maximum in Fig. 15(d).

We could develop a parallel algorithm to find the convex envelope of bivariate functions over polyhedral domains. This problem being piecewise has some level of natural parallelization and we could further partition the domain depending on the hardware available. Parallelisation of the code would require more than just using parallel for statements. We would need to implement reduction and smartly distribute data in order to get an efficient algorithm.

We could preprocess the original problem to get different divisions in order to solve the problem faster. The preprocessing step might vary depending on whether we use parallel methods.

The current examples and tests were run when the overall domain was convex. We need to run tests when the overall domain is not convex.

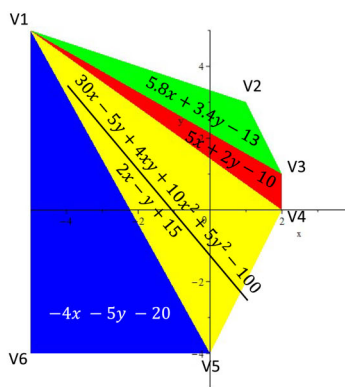
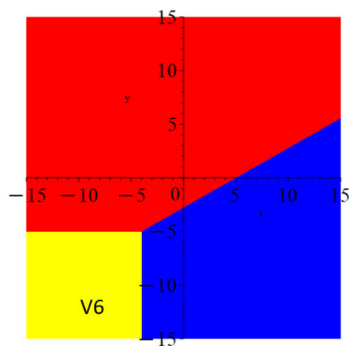
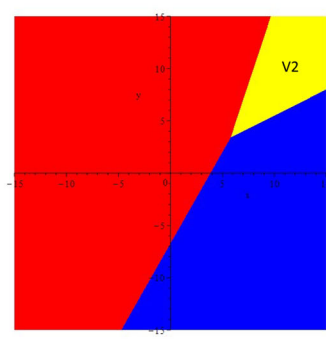
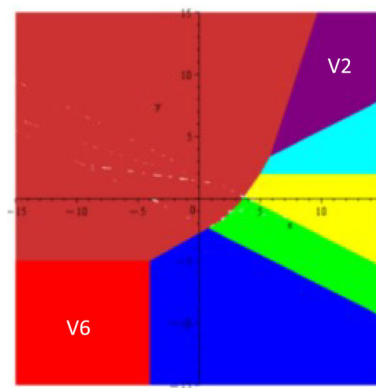
(a) $r_{1,j}$ (b) $s_{1,1,k}$ (c) $s_{1,4,k}$ (d) s_k

Fig. 15 Domain of a conjugate illustrating the potential for improvement. The conjugates corresponding to vertices and edges that are not a part of the division when computing the convex envelope of each piece appear exactly the same in the final conjugate

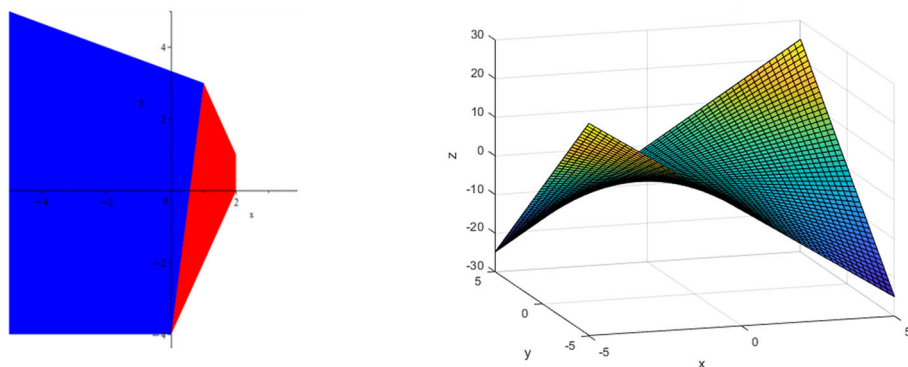


Fig. 16 Function $f(x, y) = xy$ with piecewise polyhedral domain

Appendix A Code demonstration

We demonstrate the code on a PLQ function with two pieces. First we create symbolic variables x and y and then create a function f in terms of x and y . Then we input the vertices of the polyhedral regions for the two pieces and store them in $d(1)$ and $d(2)$. We create two pieces $p(1)$ and $p(2)$, and finally the plq function.

```
x = sym('x')
y = sym('y')
f=symbolicFunction(x*y);
d(1)=domain([-5,-4;0,-4;1,3;-5,5],x,y);
d(2)=domain([0,-4;2,0;2,1;1,3],x,y);
p(1) = plq_1piece(d(1),f);
p(2) = plq_1piece(d(2),f);
example1 = plq(p);
```

Once we have the input, we can invoke the conjugate function to get the conjugate of the function. The input and output variables are the same as we are updating fields inside the class.

```
example1 = example1.conjugate
```

Now we need the output to interpret our results.

```
example1.printDomainMaple;
example1.print;
example1.printLatex;
```

Line 1 outputs the domain of the convex envelopes, conjugates and maximum conjugate in a format to generate diagrams in maple. The Figures created using this command are displayed in chapters (3-7). Line 2 outputs the convex envelope, conjugate and maximum in the MATLAB command window. Line 3 outputs the convex envelope, conjugate and maximum in $\text{\LaTeX}$ format. The example is plotted in Fig. 16.

Appendix B Symbolic verification of proofs of Proposition 4 in Section 3.1

Appendix B.1 Proof of proposition 4

The MATLAB code below verifies our computation by evaluating δ to zero.

```

m = sym('m')
q = sym('q')

psi01 = sym('psi_01');
psi02 = sym('psi_02');
psi03 = sym('psi_03');
psi11 = sym('psi_11');
psi12 = sym('psi_12');
psi13 = sym('psi_13');
psi21 = sym('psi_21');
psi22 = sym('psi_22');
psi23 = sym('psi_23');

t0 = (-psi01 - m*psi02)/(2*(psi11 + m*psi12));
t1 = 1/(2*(psi11 + m*psi12));
t2 = m/(2*(psi11 + m*psi12));
gamma10 = t1 * (psi23 + q*psi22)/(psi11 + m*psi12);
gamma01 = t2 * (psi23 + q*psi22)/(psi11 + m*psi12);
gamma00 = (t0 * (psi23 + q*psi22) - psi13 - q*(psi12))/(psi11 + m*psi12);

zeta11 = -(psi11*gamma10 + m*psi12*gamma10)^2/(psi23 + q*psi22) + gamma10;
zeta12 = -(2*(psi11*gamma01 + m*psi12*gamma01)*(psi11*gamma10 + m*psi12*gamma10))/(psi23 + q*psi22) + gamma01 + m*gamma10;
zeta22 = -(psi11 * gamma01 + m*psi12*gamma01)^2/(psi23 + q*psi22) + gamma01*m

zeta10 = -2*(psi11*gamma01 + m*psi12*gamma10)*(psi13 + psi11*gamma00 + psi12*(q + m*gamma00))/(psi23 + q*psi22) -
m*psi02*gamma10 + gamma00 - psi01*gamma10;
zeta01 = -(2*(psi11*gamma01 + m*psi12*gamma01)*(psi13 + psi11*gamma00 + psi12*(q + m*gamma00)))/((psi23 + q*psi22)) -
m*psi02*gamma01 - psi01*gamma01 + m*gamma00 + q;
zeta00 = -(psi13 + psi11*gamma00 + psi12*(q + m*gamma00))^2/(psi23 + q*psi22) - psi03 - psi01*gamma00 - psi02*(q + m*gamma00);

D = zeta12^2 - 4*zeta11*zeta22
simplifyFraction(D)

s1 = sym('s_1')
s2 = sym('s_2')

disp('quad')
fq = simplifyFraction(zeta11*s1^2 + zeta12*s1*s2 + zeta22*s2^2 + zeta10*s1 + zeta01*s2 + zeta00)

[cx, tx] = coeffs(fq, [s1, s2])

delta = simplifyFraction(cx(2)^2 - 4*cx(1)*cx(4))

```

Appendix B.2 Proof of zero denominator at one vertex

```

a= sym('a')
b= sym('b')
m= sym('m')
q= sym('q')
fv = sym('fv')
xv = sym('x_1')
yv = sym('y_1')
fv = xv*yv
dl= sym('dl')
du= sym('du')
etah = -(a+m*b-q)^2/(4*m)-b*q
etaw = fv - a*xv - b*yv
eq = etah - etaw
z = sym('z')
eq2 = subs(eq, a, z-m*b)
bv = solve(eq2,b)
obj = etaw + a*x+ b*y
obj = subs(obj, a, z-m*b)
obj = subs(obj, b, bv)
[cz,terms] = coeffs(obj,z)
psi2 = -simplify(cz(1) )
psi1 = simplify(cz(2) )/2
psi0 = simplify(cz(3) )
subs(psi2, [x,y], [xv,yv])
subs(psi1, [x,y], [xv,yv])
obj1 = simplifyFraction(psi1^2/psi2 + psi0)
obj2 = simplifyFraction(-psi2*dl^2+2*dl*psi1+psi0)
obj2 = simplifyFraction(-psi2*du^2+2*du*psi1+psi0)

```

Acknowledgements This work was supported by Discovery grant RGPIN-2018-03928 (second author) from the Natural Sciences and Engineering Research Council of Canada (NSERC).

Author contributions Not applicable.

Funding This work was supported by Discovery grant RGPIN-2018-03928 (second author) from the Natural Sciences and Engineering Research Council of Canada (NSERC).

Data Availability Not applicable.

Code availability All code and examples are available at <https://github.com/tanmaya11/convex/tree/biconjugate2>.

Declarations

Conflict of Interest The authors declare that they have no Conflict of interest.

Ethics approval and consent to participate Not applicable.

Consent for publication Not applicable.

Materials availability Not applicable.

References

1. Locatelli, M., Schoen, F.: Global Optimization. MOS-SIAM Series on Optimization, p. 432. Society for Industrial and Applied Mathematics, Philadelphia, PA (2013). <http://epubs.siam.org/doi/abs/10.1137/1.9781611972672.fm>
2. Crama, Y.: Recognition problems for special classes of polynomials in 0–1 variables. *Mathematical Programming* **44**, 139–155 (1989)
3. Locatelli, M., Schoen, F.: On convex envelopes for bivariate functions over polytopes. *Mathematical Programming* **144**(1–2, Ser. A), 65–91 (2014). <https://doi.org/10.1007/s10107-012-0616-x>
4. Locatelli, M.: A technique to derive the analytical form of convex envelopes for some bivariate functions. *Journal of global optimization* **59**(2), 477–501 (2014). <https://doi.org/10.1007/s10898-014-0177-z>
5. Locatelli, M.: Polyhedral subdivisions and functional forms for the convex envelopes of bilinear, fractional and other bivariate functions over general polytopes. *Journal of global optimization* **66**(4), 629–668 (2016). <https://doi.org/10.1007/s10898-016-0418-4>
6. Locatelli, M.: Convex envelopes of bivariate functions through the solution of KKT systems. *Journal of global optimization* **72**(2), 277–303 (2018). <https://doi.org/10.1007/s10898-018-0626-1>
7. Khademnia, E., Davarnia, D.: Convexification of bilinear terms over network polytopes. *Mathematics of operations research* (2024). <https://doi.org/10.1287/moor.2023.0001>
8. Al-Khayyal, F., Falk, J.: Jointly constrained biconvex programming. *Mathematics of Operations Research* **8** (1983)
9. Sherali, H., Alameddine, A.: An explicit characterization of the convex envelope of a bivariate bilinear function over special polytopes. *Annals of Operations Research* **25**, 197–209 (1990)
10. Linderoth, J.: A simplicial branch-and-bound algorithm for solving quadratically constrained quadratic programs. *Mathematical Programming* **2**, 251–282 (2005)
11. Anstreicher, K.M., Burer, S.: Computable representations for convex hulls of low-dimensional quadratic forms. *Mathematical programming* **124**(1–2), 33–43 (2010). <https://doi.org/10.1007/s10107-010-0355-9>
12. Anstreicher, K.: On convex relaxations for quadratically constrained quadratic programming. *Mathematical Programming* **136**, 233–251 (2012)
13. Locatelli, M.: A new technique to derive tight convex underestimators (sometimes envelopes). *Computational optimization and applications* **87**(2), 475–499 (2024). <https://doi.org/10.1007/s10589-023-00534-8>
14. Rockafellar, R.T., Wets, R.J.-B.: Variational Analysis. Springer, ??? (1998). <http://www.springer.com/math/book/978-3-540-62772-2>
15. Goebel, R.: Convexity, convergence and feedback in optimal control. PhD thesis, University of Washington, Seattle (2000). <https://digital.lib.washington.edu/dspace/handle/1773/5792>
16. Gardiner, B., Jakee, K., Lucet, Y.: Computing the partial conjugate of convex piecewise linear-quadratic bivariate functions. *Computational Optimization and Applications* **58**(1), 249–272 (2014). <https://doi.org/10.1007/s10589-013-9622-z>
17. Moreau, J.-J.: Proximité et dualité dans un espace Hilbertien. *Bull. Soc. Math. France* **93**, 273–299 (1965)
18. Brenier, Y.: Un algorithme rapide pour le calcul de transformées de Legendre–Fenchel discrètes. *C. R. Acad. Sci. Paris Sér. I Math.* **308**, 587–589 (1989)
19. Corrias, L.: Fast Legendre–Fenchel transform and applications to Hamilton–Jacobi equations and conservation laws. *SIAM J. Numer. Anal.* **33**(4), 1534–1558 (1996)
20. Lucet, Y.: A fast computational algorithm for the Legendre–Fenchel transform. *Computational optimization and Applications* **6**(1), 27–57 (1996)
21. Lucet, Y.: Faster than the Fast Legendre Transform, the Linear-time Legendre Transform. *Numerical Algorithms* **16**(2), 171–185 (1997)
22. Haque, T., Lucet, Y.: A linear-time algorithm to compute the conjugate of convex piecewise linear-quadratic bivariate functions. *Computational Optimization and Applications* **70**(2), 593–613 (2018). <https://doi.org/10.1007/s10589-018-0007-1>
23. Lucet, Y., Bauschke, H.H., Trienis, M.: The piecewise linear-quadratic model for computational convex analysis. *Computational Optimization and Applications* **43**, 95–118 (2009)
24. Gardiner, B., Lucet, Y.: Graph-matrix calculus for computational convex analysis. In: Bauschke, H.H., Burachik, R.S., Combettes, P.L., Elser, V., Luke, D.R., Wolkowicz, H. (eds.) *Fixed-Point Algorithms for Inverse Problems in Science and Engineering*. Springer Optimization and Its Applications, vol. 49, pp. 243–259. Springer, ??? (2011). https://doi.org/10.1007/978-1-4419-9569-8_12
25. Lucet, Y.: Computational Convex Analysis (CCA) numerical library. GitHub (2017)
26. Lucet, Y.: Computational Convex Analysis for bivariate piecewise linear-cubic functions (CCA2) numerical library. GitHub (2021)

27. Hiriart-Urruty, J.-B., Lucet, Y.: Parametric computation of the Legendre-Fenchel conjugate. *Journal of Convex Analysis* **14**(3), 657–666 (2007)
28. Lucet, Y.: Fast Moreau envelope computation I: Numerical algorithms. *Numer. Algorithms* **43**(3), 235–249 (2006). <https://doi.org/10.1007/s11075-006-9056-0>
29. Bauschke, H.H., Goebel, R., Lucet, Y., Wang, X.: The proximal average: Basic theory. *SIAM J. Optim.* **19**(2), 768–785 (2008)
30. Bauschke, H.H., Lucet, Y., Trienis, M.: How to transform one convex function continuously into another. *SIAM Rev.* **50**(1), 115–132 (2008)
31. Singh, S., Lucet, Y.: Linear-time convexity test for low-order piecewise polynomials. *SIAM Journal on Optimization* **31**(1), 972–990 (2021). <https://doi.org/10.1137/19M1290851>
32. Lucet, Y.: Computational Convex Analysis library 2 (2021). <https://github.com/ylucet/CCA2>
33. Lucet, Y.: What shape is your conjugate? A survey of computational convex analysis and its applications. *SIAM Review* **52**(3), 505–542 (2010)
34. Kumar, D., Lucet, Y.: Towards the biconjugate of bivariate piecewise quadratic functions. In: Le Thi, H.A., Le, H.M., Pham Dinh, T. (eds.) *Optimization of Complex Systems: Theory, Models, Algorithms and Applications*, pp. 257–266. Springer, Cham (2020)
35. Rockafellar, R.T.: On the maximal monotonicity of subdifferential mappings. *Pacific J. Math.* **33**, 209–216 (1970)
36. Patrinos, P., Sarimveis, H.: Convex parametric piecewise quadratic optimization: Theory and algorithms. *Automatica* **47**(8), 1770–1777 (2011)
37. Gardiner, B., Lucet, Y.: Computing the conjugate of convex piecewise linear-quadratic bivariate functions. *Mathematical Programming* **139**(1–2), 161–184 (2013). <https://doi.org/10.1007/s10107-013-0666-8>
38. Bauschke, H.H., Moursi, W.M.: *An Introduction to Convexity, Optimization, and Algorithms*, vol. 34. Society for Industrial and Applied Mathematics, Philadelphia (2024)
39. Hiriart-Urruty, J.-B., Lemaréchal, C.: *Convex Analysis and Minimization Algorithms I. Grundlehren der Mathematischen Wissenschaften [Fundamental Principles of Mathematical Sciences]*, vol. 305. Springer, Berlin (1993). Vol I: Fundamentals. <http://www.springer.com/math/book/978-3-540-56850-6>
40. Hiriart-Urruty, J.-B., Lemaréchal, C.: *Convex Analysis and Minimization Algorithms II. Grundlehren der Mathematischen Wissenschaften [Fundamental Principles of Mathematical Sciences]*, vol. 306. Springer, Berlin (1993). Vol II: Advanced theory and bundle methods. <http://www.springer.com/math/book/978-3-540-56850-6>
41. Rockafellar, R.T.: *Convex Analysis*. Princeton University Press, ??? (1970). <http://press.princeton.edu/titles/1815.html>
42. Kumar, D.: Conjugate of some rational functions and convex envelope of quadratic functions over a polytope. Master's thesis, University of British Columbia (2019)
43. Karmarkar, T.: Computing the conjugate of nonconvex bivariate piecewise linear-quadratic functions. Master's thesis, University of British Columbia (2024). <https://doi.org/10.14288/1.0444097>
44. Chan, T.M.: A simpler linear-time algorithm for intersecting two convex polyhedra in three dimensions. *Discrete & Computational Geometry* **56**(4), 860–865 (2016). <https://doi.org/10.1007/s00454-016-9785-3>
45. Berg, M., Kreveld, M., Overmars, M., Schwarzkopf, O.: *Computational Geometry*, 3rd edn., p. 386. Springer, Berlin (2008). Algorithms and applications. <http://www.springer.com/computer/theoretical+computer+science/book/978-3-540-77973-5>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.